GIẢI PHÁP ỨNG DỤNG AI TRONG ĐÁNH GIÁ TRỰC TUYẾN HỆ THỐNG HOUSCORE-AI

MỤC LỤC

GIẢI PHÁP ỨNG DỤNG AI TRONG ĐÁNH GIÁ TRỰC TUYẾN 1

HỆ THỐNG HOUSCORE-AI 1

1. Giới thiệu 4

1.1. Mục đích 4

1.2. Phạm vi hệ thống 4

1.3. Thuật ngữ 4

2. Các yêu cầu chức năng 5

2.1. Các tác nhân 5

2.2. Các chức năng của hệ thống 5

2.3. Biểu đồ Use Case tổng quát 7

2.4. Biểu đồ Use Case chi tiết 8

2.5. Quy trình nghiệp vụ 12

2.6. Đặc tả các Use Case chi tiết 18

3. Các yêu cầu phi chức năng 31

3.1. Khả năng thực thi – Performance 31

3.2. Bảo mật – Security 31

3.3. Khả năng vận hành 32

4. Thiết kế hệ thống 32

4.1. Kiến trúc tổng thể 32

4.2. Thiết kế lớp dữ liệu 34

4.3. Thiết kế luồng xử lý bất đồng bộ 35

4.4. Thiết kế tích hợp và bảo mật 35

4.5. Thiết kế AI Core 37

4.6. Thiết kế API và giao tiếp giữa các service 38

4.7. Thiết kế giao diện người dùng 39
4.8. Thiết kế cơ sở dữ liệu 41
4.9. Thiết kế bảo mật chi tiết 46
4.10. Lựa chọn công nghệ 48

DANH MỤC BẢNG

Bảng 1. Đặc tả UC Quản lý khóa học 18
Bảng 2. Đặc tả UC Quản lý tài khoản giảng viên 19
Bảng 3. Đặc tả UC Quản lý Rubric 21
Bảng 4. Đặc tả UC Quản lý kho dữ liệu 22
Bảng 5. Đặc tả UC Tải bài làm Submission 23
Bảng 6. Đặc tả UC Chạy phân tích AI 24
Bảng 7. Đặc tả UC Hiệu chỉnh điểm và nhận xét 26
Bảng 8. Đặc tả UC Công bố kết quả 27
Bảng 9. Đặc tả UC Xem kết quả chi tiết bài làm 28
Bảng 10. Đặc tả UC Cấu hình tích hợp LMS 29
Bảng 11. NFR Khả năng thực thi 31
Bảng 12. NFR Bảo mật 31
Bảng 13. NFR Khả năng vận hành 32
Bảng 14. Bảng mô tả kiến trúc tổng thể HouScore-AI 33
Bảng 15. Thiết kế lớp dữ liệu 34
Bảng 16. Tích hợp hệ thống LMS 35
Bảng 17.API Endpoint (ví dụ Core Service) 39
Bảng 18. Bảng `users` (Quản lý Tài khoản Giảng viên - UC-09b) 41
Bảng 19. Bảng `lms_integrations` (Cấu hình Tích hợp LMS - UC-10) 41
Bảng 20. Bảng `courses` (Quản lý Khóa học - UC-09a) 42
Bảng 21. Bảng `rubrics` (Quản lý Rubric - UC-06) 43
Bảng 22. Bảng `rubric_criteria` (Chi tiết Rubric - UC-06) 43

Bảng 23. Bảng `audit_logs` (Kiểm tra Truy vết - FR-3.2, UC-08) 44

DANH MỤC HÌNH

Hình 1. Quy trình cấu hình tham số 13

Hình 2. Quy trình tải dữ liệu – data ingestion 14

Hình 3. Quy trình khởi động xử lý bất đồng bộ 15

Hình 4. Quy trình thực thi AI 16

Hình 5. Quy trình quản lý kết quả và hiệu chỉnh 17

Hình 6. Quy trình phản hồi và phân phối 18

Hình 7. Mô hình Microservices 33

DANH MỤC BIỂU ĐỒ

Biểu đồ 1. Use Case tổng quát Hệ thống HouScore-AI 8

Biểu đồ 2. Use Case Quản lý khóa học 8

Biểu đồ 3. Use Case Quản lý tài khoản giảng viên 8

Biểu đồ 4. Use Case Quản lý Rubric 9

Biểu đồ 5. Use Case Quản lý Kho dữ liệu 9

Biểu đồ 6. Use Case Tải bài làm 10

Biểu đồ 7. Use Case chạy phân tích AI 10

Biểu đồ 8. Use Case Hiệu chỉnh điểm, nhận xét 11

Biểu đồ 9. Use Case Công bố kết quả 11

Biểu đồ 10. Use Case Xem kết quả chi tiết bài làm 12

Biểu đồ 11. Use Case Cấu hình tích hợp LMS 12

1. Giới thiệu

1.1. Mục đích

Xác định các yêu cầu chức năng và phi chức năng chi tiết cho việc phát triển hệ thống HouScore-AI. Hệ thống nhằm mục đích triển khai các giải pháp học sâu (Deep Learning) và xử lý ngôn ngữ tự nhiên (NLP) để kiến tạo một công cụ hồi quy ngữ nghĩa nhằm tự động hóa quy trình chấm điểm bài luận, tăng cường khả năng phát hiện gian lận dựa trên so sánh vector ngữ nghĩa, cung cấp luồng phản hồi chi tiết, có cấu trúc đến người học.

1.2. Phạm vi hệ thống

HouScore-AI được thiết kế dưới dạng ứng dụng web độc lập với kiến trúc Microservices, ưu tiên khả năng mở rộng và tích hợp thông qua API. Hệ thống tập trung vào module xử lý backend cho các tác vụ tốn tài nguyên. Các module chức năng cốt lõi bao gồm:

- Data Ingestion & Preprocessing: Hỗ trợ tiếp nhận và chuẩn hóa các định dạng tài liệu đầu vào (docx, pdf, txt) thành chuỗi văn bản đã được tiền xử lý (tokenized, normalized).
- AI core scoring: Triển khai mô hình hồi quy, phân loại văn bản dựa trê Rubric và trọng số được giảng viên cấu hình.
- Semantic plagiarism detection: Sử dụng kỹ thuật nhúng từ (word embeddings) và so sánh ngữ nghĩa (semantic similarity) để xác định tỷ lệ trùng lặp.
- Asynchronous feedback generation: Tự động tạo và phân phối các gói dữ liệu kết quả bao gồm: điểm số cuối cùng, nhận xét từ NLP, báo cáo đạo văn tới các kênh phân phối như email service, LMS API.

1.3. Thuật ngữ

Thuật ngữ	Định nghĩa
AI Core	Module backend (microservice) chứa các mô hình học sâu đjc huấn luyện để thực hiện chấm điểm và phát hiện đạo văn.
Deep learning	Phương pháp ML sử dụng mạng lưới nơ-ron sâu (DNN) để thực hiện các tác vụ phức tạp như phân loại, hồi quy văn bản.

Rubric weighting	Trọng số được gán cho các tiêu chí chấm điểm, là tham số đầu vào quan trọng để huấn luyện mô hình hoặc tinh chỉnh kết quả cuối cùng.
LTI	Learning tools interoperability. Một chuẩn API được ưu tiên sử dụng để tích hợp an toàn với các LMS.
Vector Embedding	Biểu diễn ngữ nghĩa của từ hoặc tài liệu dưới dạng vector mật độ cao trong không gian nhiều chiều.
FR/NFR	Functional Requirement/ Non- Functional Requirement (Yêu cầu chức năng/ Yêu cầu phi chức năng)

2. Các yêu cầu chức năng

2.1. Các tác nhân

Tác nhân (Actor)	Mô tả
Giảng viên	Là tác nhân chính của hệ thống, chịu trách nhiệm quản lý và vận hành quy trình đánh giá.
Sinh viên	là tác nhân của hệ thống, tương tác với hệ thống để xem kết quả đánh giá bài làm của mình
Hệ thống LMS	là hệ thống bên ngoài, tương tác với hệ thống HouScore-AI để trao đổi dữ liệu một cách tự động, nâng cao hiệu quả làm việc.

2.2. Các chức năng của hệ thống

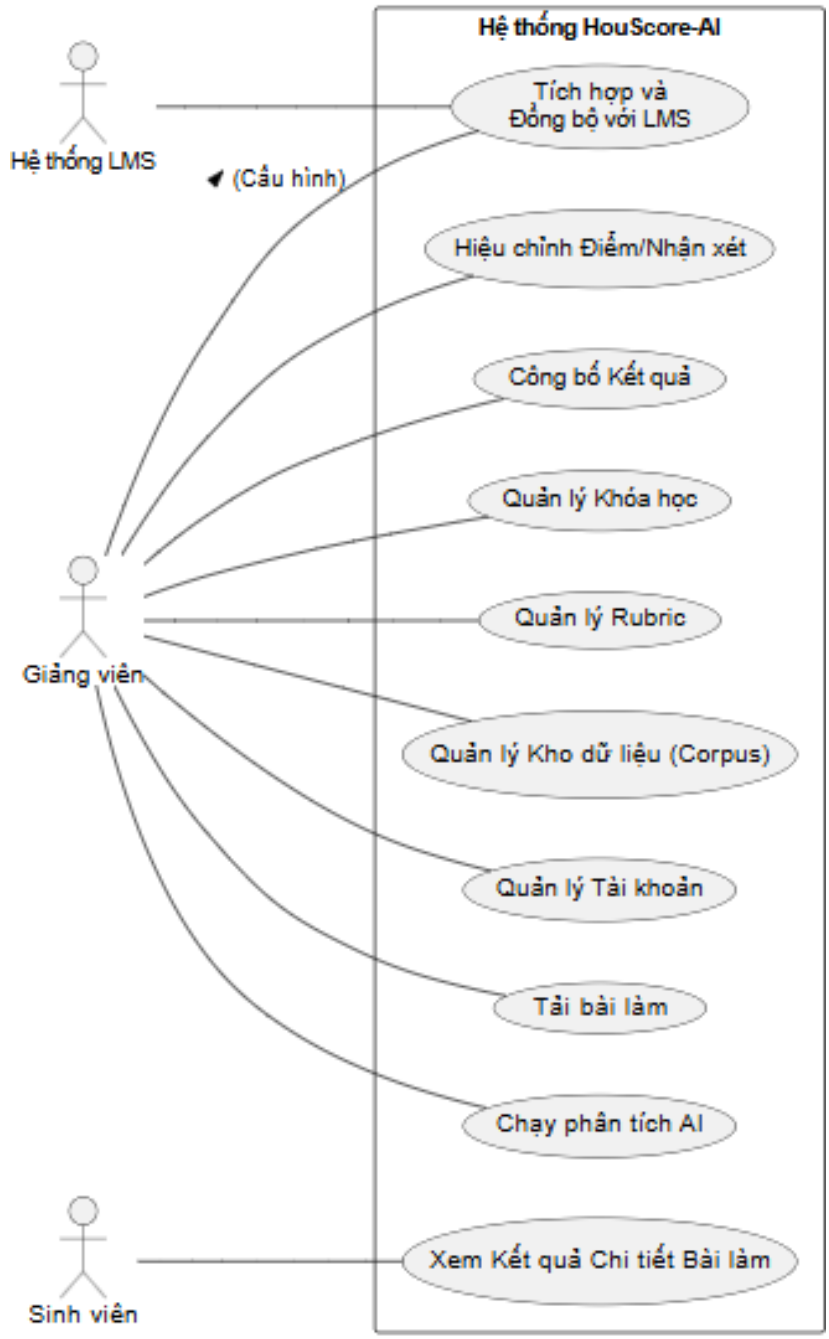
Các yêu cầu chức năng mô tả các hành vi và chức năng cụ thể mà hệ thống phải thực hiện để đáp ứng nhu cầu của người dùng.

ID	Yêu cầu Chức năng	Mô tả Chi tiết
FR-1.1	Tiếp nhận và Tiền xử lý File	Hệ thống phải tiếp nhận và xử lý tệp bài làm đầu vào (DOCX, PDF, TXT) bằng cách trích xuất văn bản thuần túy và làm sạch dữ liệu trước khi chuyển cho AI Core.

FR-1.2	Chấm điểm Tự động (AI Scoring)	Hệ thống phải thực hiện tính toán điểm dự đoán cho từng tiêu chí dựa trên Rubric và mô hình Học máy (ML) đã huấn luyện.
FR-1.3	Phát hiện Đạo văn Ngữ nghĩa	Hệ thống phải thực hiện so sánh ngữ nghĩa (Semantic Search) giữa bài làm với Kho dữ liệu Corpus (UC-07) và các bài làm khác để xác định và đánh dấu các đoạn bị trùng lặp.
FR-2.1	Thiết lập Trọng số Rubric	Hệ thống phải cho phép Giảng viên định nghĩa các tiêu chí Rubric và thiết lập trọng số (Weight) sao cho **tổng trọng số bằng 100%**.
FR-2.2	Quản lý Corpus (Vectorization)	Hệ thống phải chuyển đổi các tài liệu Corpus đã tải lên thành **Vector Embeddings** và lưu trữ chúng trong Vector DB để phục vụ Semantic Search.
FR-3.1	Quản lý Tài khoản & Khóa học	Hệ thống phải cung cấp chức năng quản lý thông tin tài khoản Giảng viên và Quản lý vòng đời của Khóa học (Tạo/Sửa/Xóa/Đồng bộ).
FR-3.2	Ghi Log Audit Trail	Mọi hành động hiệu chỉnh điểm số hoặc trạng thái đạo văn thủ công của Giảng viên **bắt buộc phải** được ghi lại vào bản ghi **AuditLog** (không thể sửa đổi) cùng với lý do hiệu chỉnh.
FR-4.1	Ghi nhận Điểm Chính thức	Hệ thống phải cập nhật trạng thái bài làm thành PUBLISHED và ghi nhận điểm số cuối cùng (ScoreFinal) là điểm chính thức của Sinh viên.
FR-4.2	Gửi Thông báo	Hệ thống phải tự động tạo và gửi thông báo (email/LMS) chứa kết quả và liên kết an toàn đến Sinh viên ngay sau khi Giảng viên công bố.

FR-4.3	Hiển thị Phản hồi NLP Chi tiết	Hệ thống phải hiển thị kết quả chấm điểm dưới dạng báo cáo chi tiết, bao gồm cả các phản hồi được tạo ra bởi mô hình Generative AI (Google Gemini) theo cấu trúc đã định.
FR-5.1	Đồng bộ Điểm với LMS (Grade Passback)	Hệ thống phải tự động gửi điểm số cuối cùng (ScoreFinal) tới Gradebook của Hệ thống LMS đã được cấu hình tích hợp.

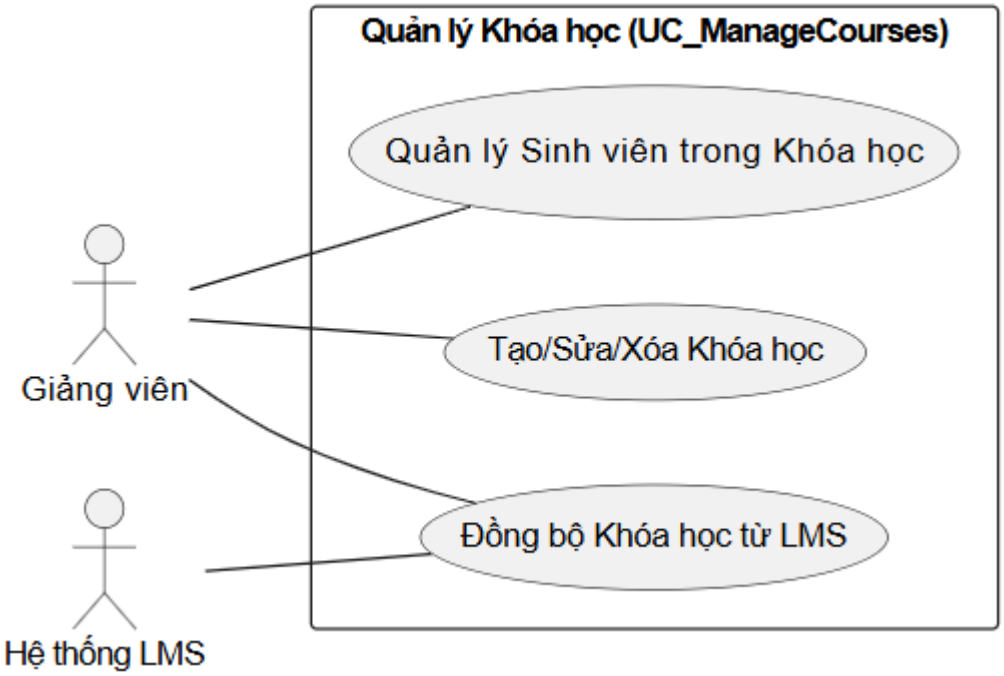
2.3. Biểu đồ Use Case tổng quát



Biểu đồ 1. Use Case tổng quát Hệ thống HouScore-AI

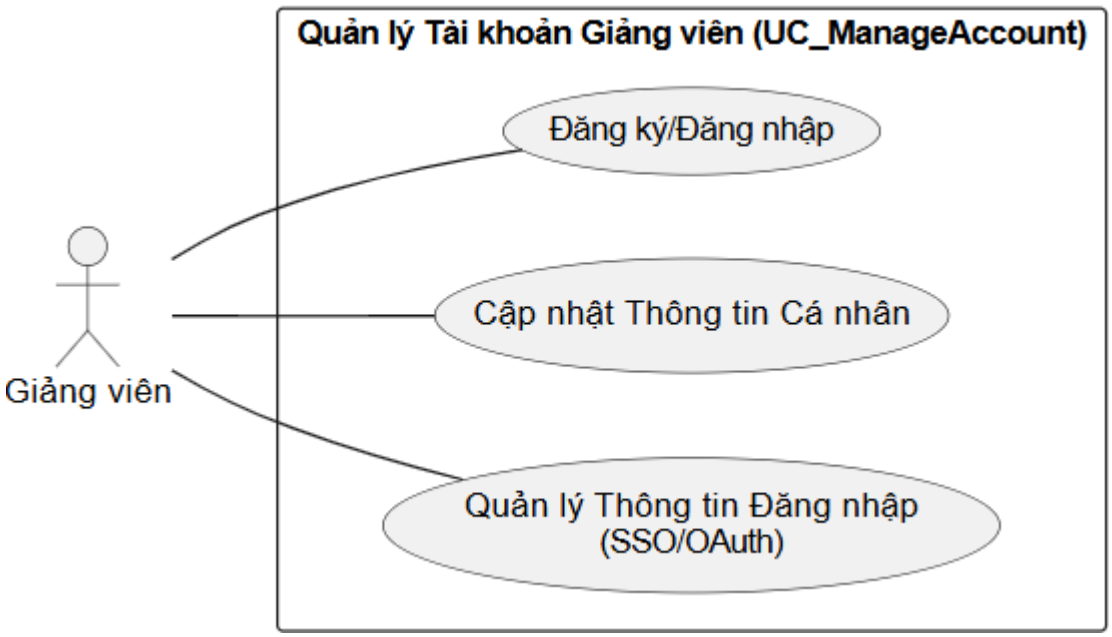
2.4. Biểu đồ Use Case chi tiết

2.4.1. Use Case Quản lý khóa học



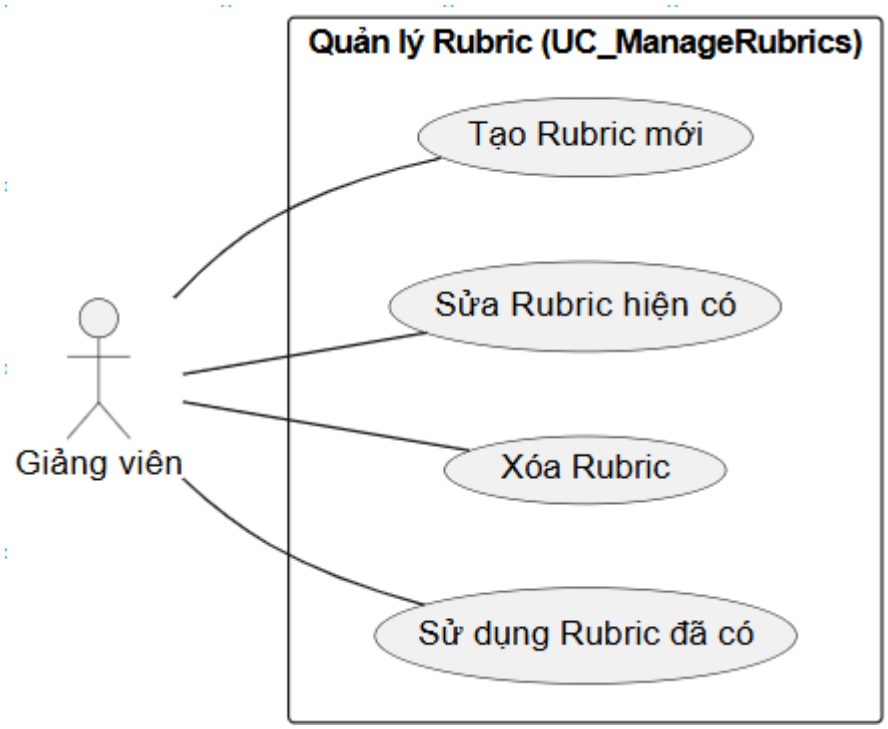
Biểu đồ 2. Use Case Quản lý khóa học

2.4.2. Use Case Quản lý tài khoản giảng viên



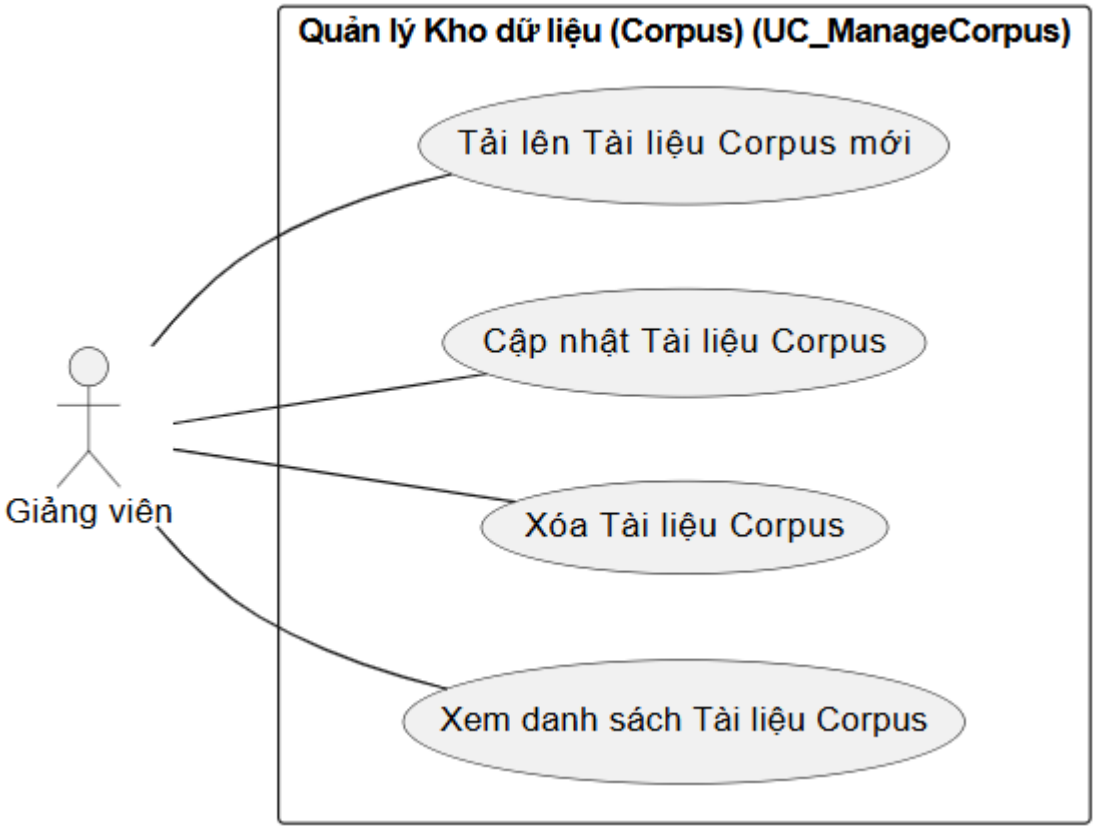
Biểu đồ 3. Use Case Quản lý tài khoản giảng viên

2.4.3. Use Case Quản lý Rubric



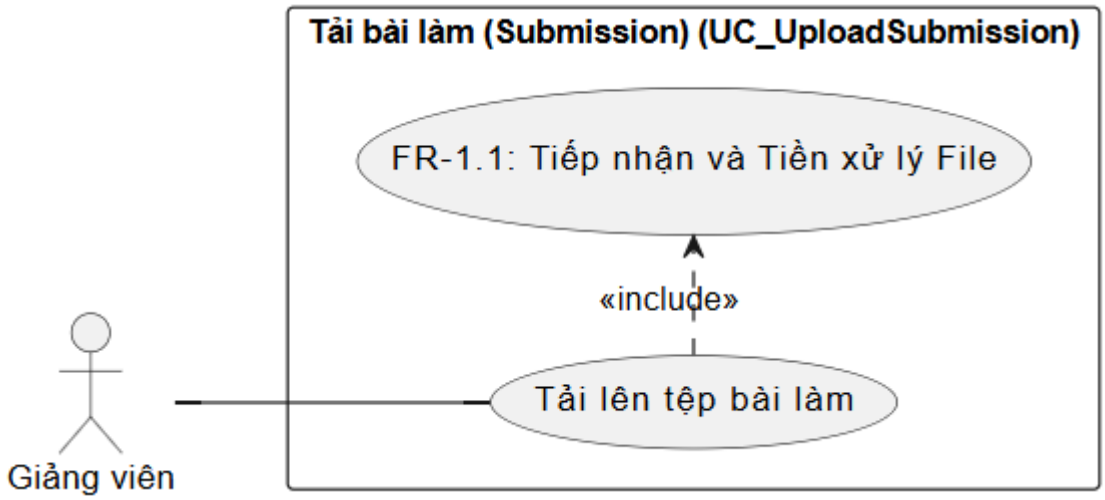
Biểu đồ 4. Use Case Quản lý Rubric

2.4.5. Use Case Quản lý Kho dữ liệu



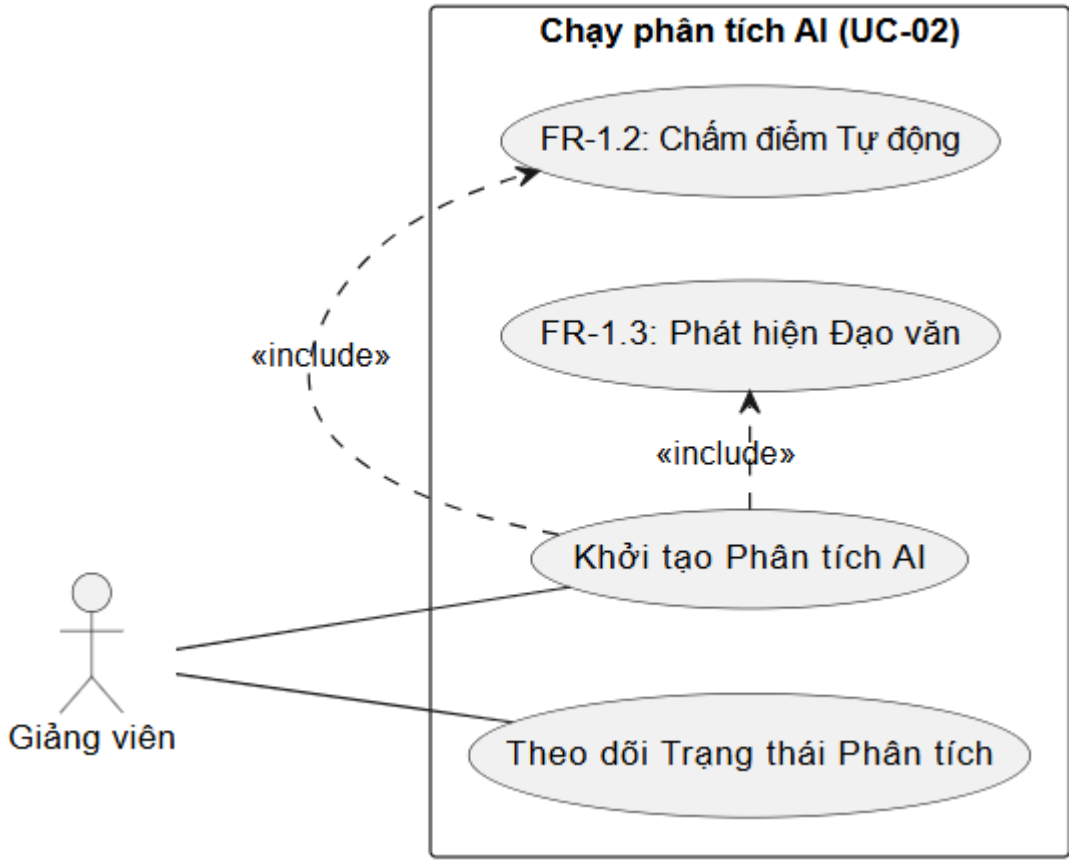
Biểu đồ 5. Use Case Quản lý Kho dữ liệu

2.4.6. Use Case Tải bài làm



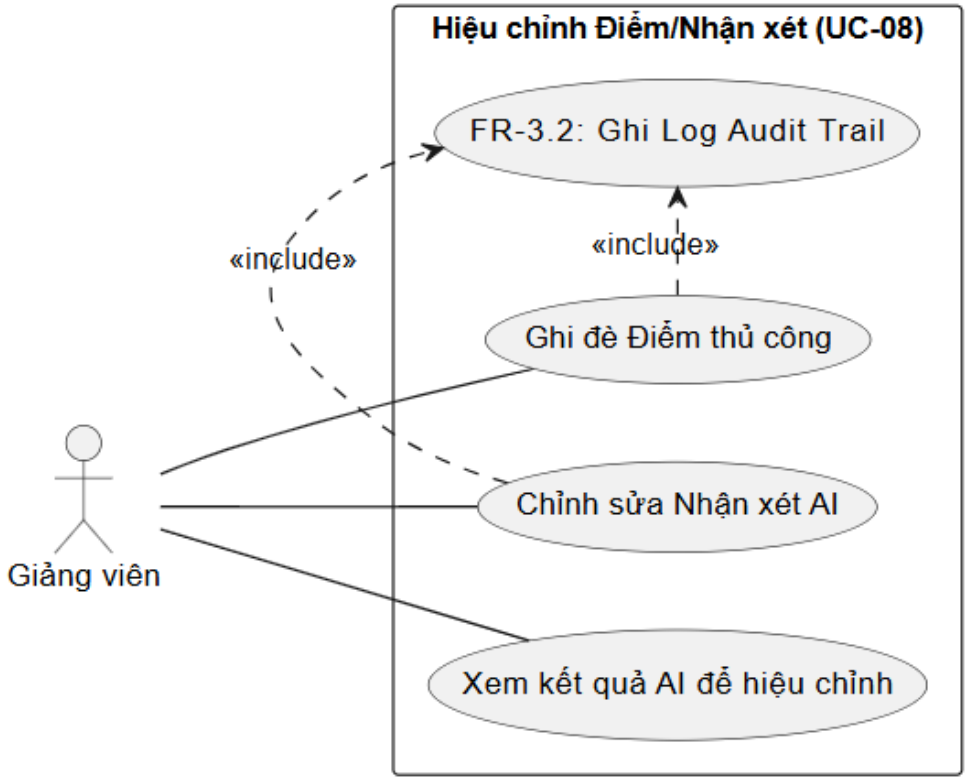
Biểu đồ 6. Use Case Tải bài làm

2.4.7. Use Case Chạy phân tích AI



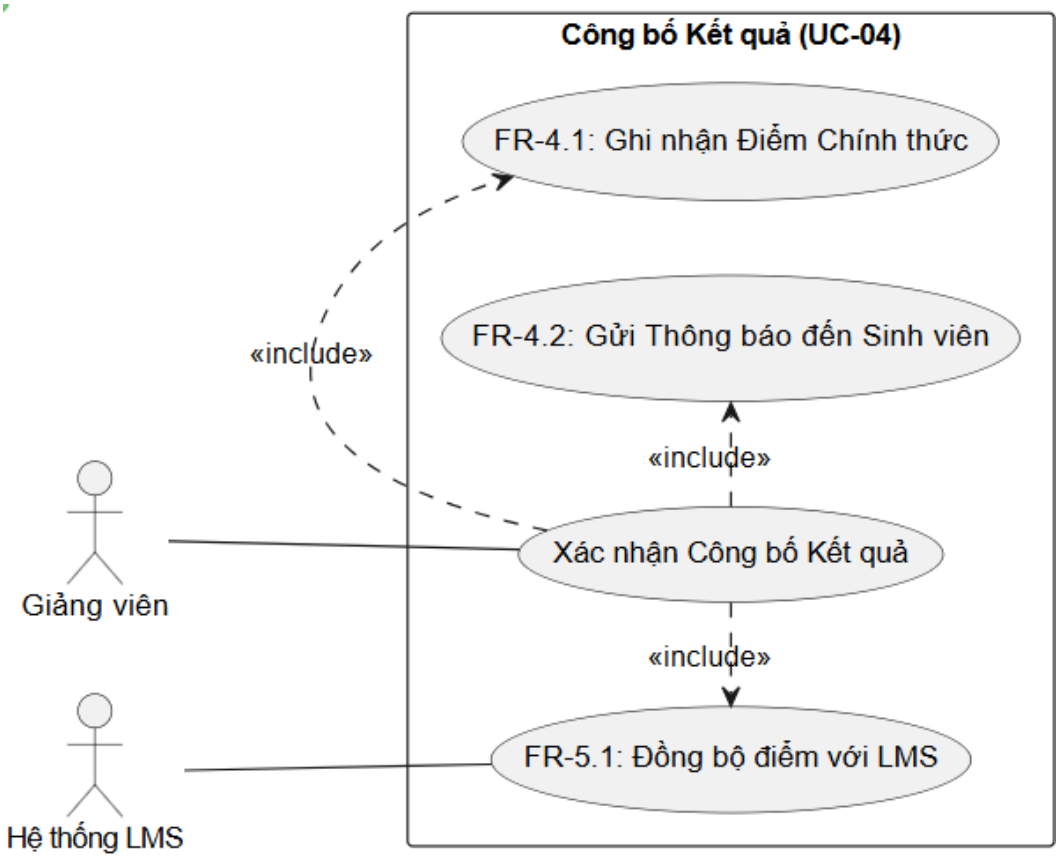
Biểu đồ 7. Use Case chạy phân tích AI

2.4.8. Use Case Hiệu chỉnh điểm, nhận xét



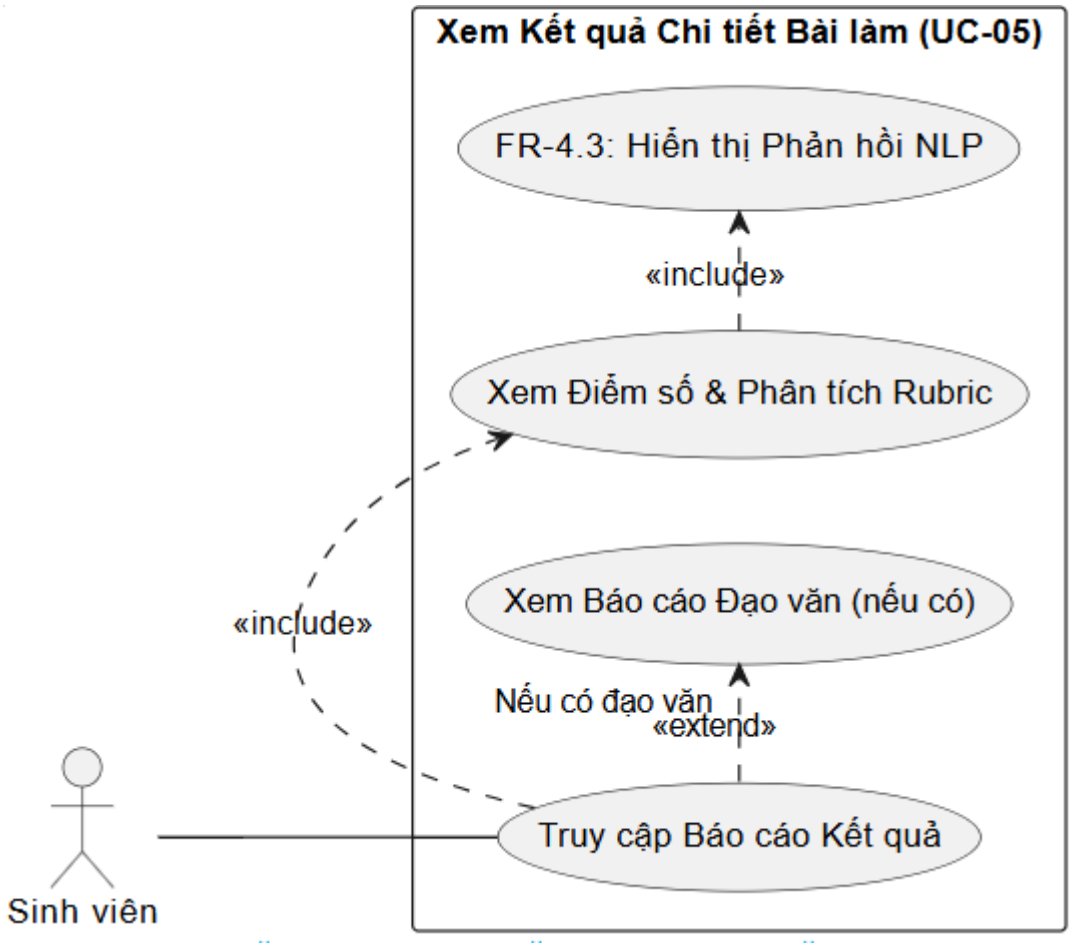
Biểu đồ 8. Use Case Hiệu chỉnh điểm, nhận xét

2.4.9. Use Case Công bố kết quả



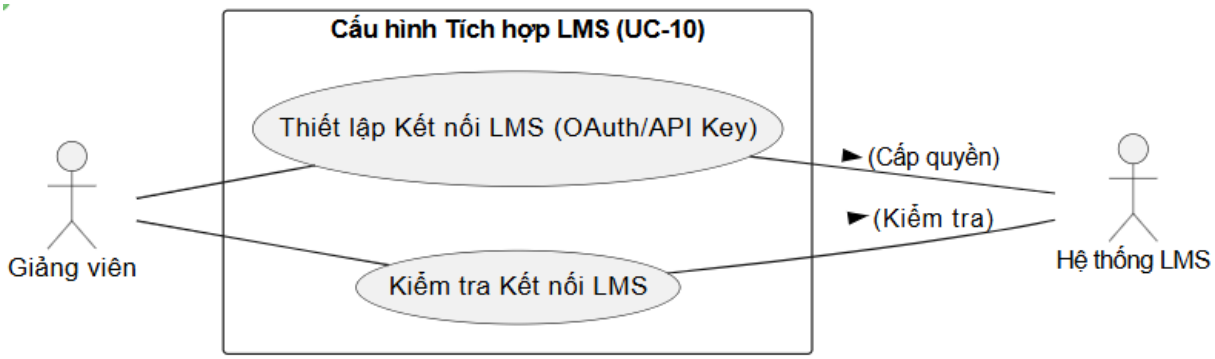
Biểu đồ 9. Use Case Công bố kết quả

2.4.10. Use Case Xem kết quả chi tiết bài làm



Biểu đồ 10. Use Case Xem kết quả chi tiết bài làm

2.4.11. Use Case Cấu hình tích hợp LMS



Biểu đồ 11. Use Case Cấu hình tích hợp LMS

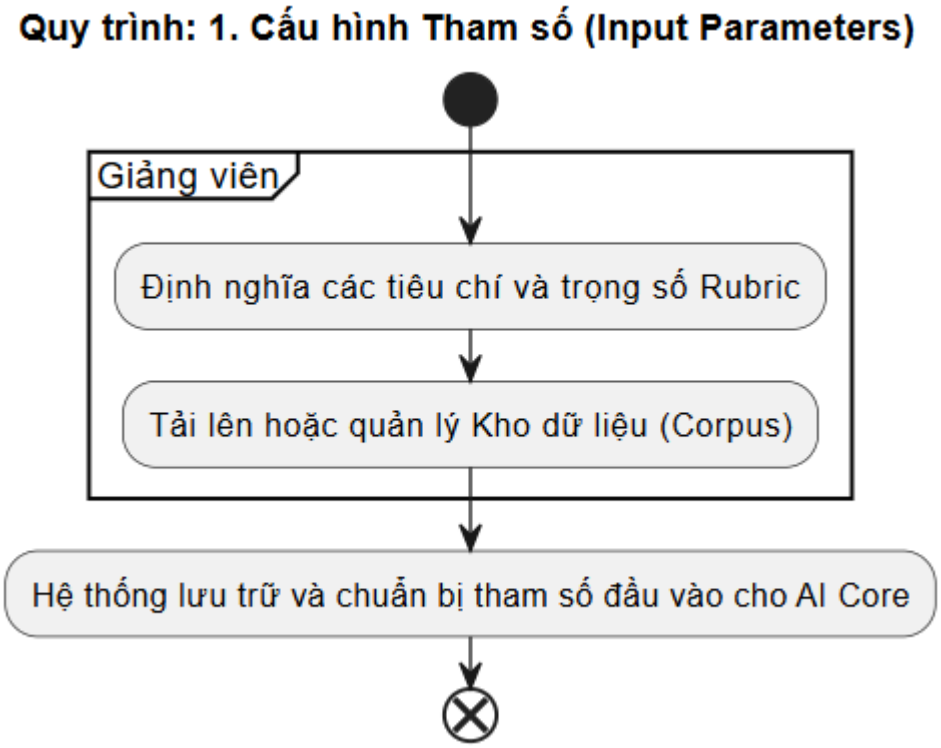
2.5. Quy trình nghiệp vụ

2.5.1. Quy trình cấu hình tham số

Mục đích của quy trình là chuẩn bị và thiết lập các yếu tố đầu vào cần thiết cho quá trình phân tích AI. Bao gồm các bước chính sau:

(1) Định nghĩa Rubric: giảng viên sử dụng giao diện hệ thống để tạo hoặc chọn một Rubric, thiết lập các tiêu chí và chỉ định trọng số cho mỗi tiêu chí, đảm bảo đôgnr trọng số bằng 100%.

(2) Quản lý corpus: Giảng viên tải lên các tài liệu tham khảo nội bộ. Data ingestion service xử lý các tệp này và gửi cho AI core service để chuyển đổi thành vector embeddings lưu trữ trong vector DB.



Hình 1. Quy trình cấu hình tham số

2.5.2. Quy trình tải dữ liệu – Data ingestion

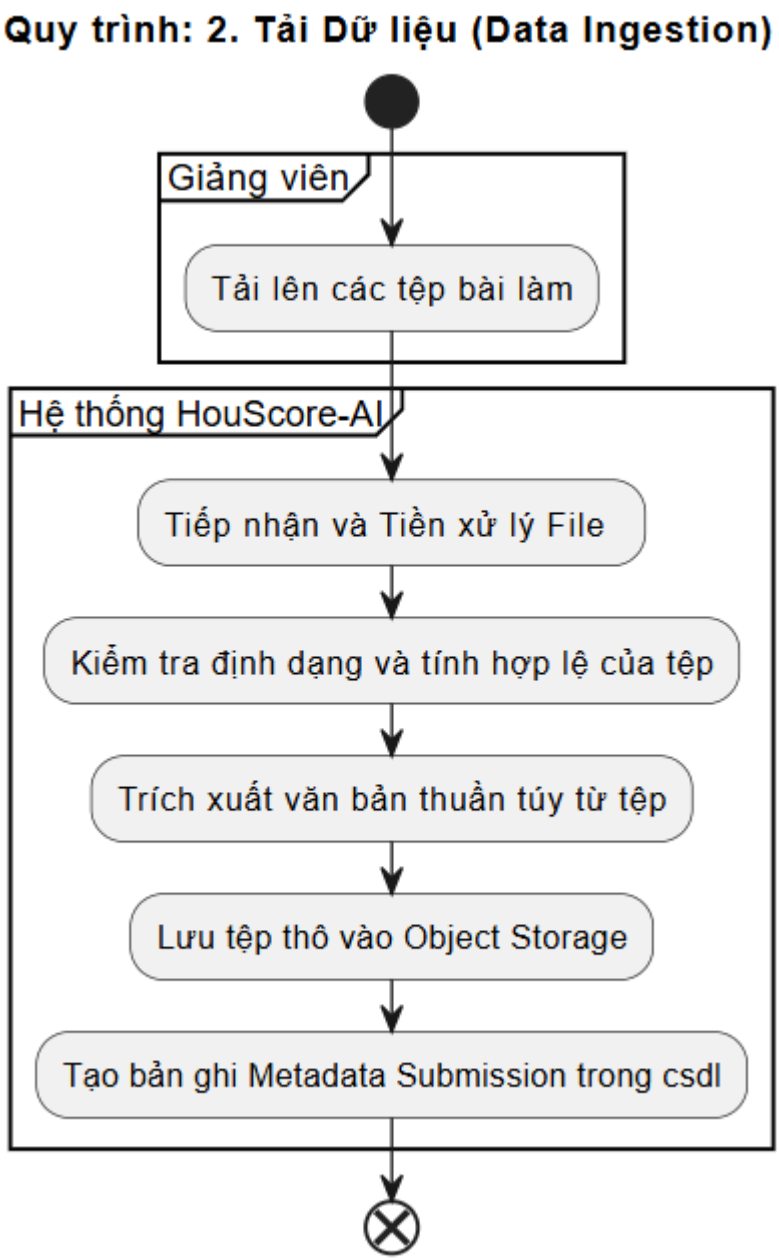
Mục đích của quy trình là đưa các tệp bài làm thô vào hệ thống một cách an toàn và chuẩn bị văn bản cho phân tích. Bao gồm các bước chính sau:

(1) Tải lên: Giảng viên tải các tệp bài làm hỗ trợ các định dạng file .docx, .pdf, .txt qua API Gateway.

(2) Tiếp nhận và tiền xử lý: data ingestion service tiếp nhận tệp, kiểm tra tính hợp lệ và loại bỏ các thành phần không phải văn bản như hình ảnh, định dạng…

(3) Lưu trữ: lưu tệp thô vào Object Storage.

(4) Metadata: core service tạo bản ghi metadata, ghi lại đường dẫn.



Hình 2. Quy trình tải dữ liệu – data ingestion

2.5.3. Quy trình khởi động xử lý bất đồng bộ

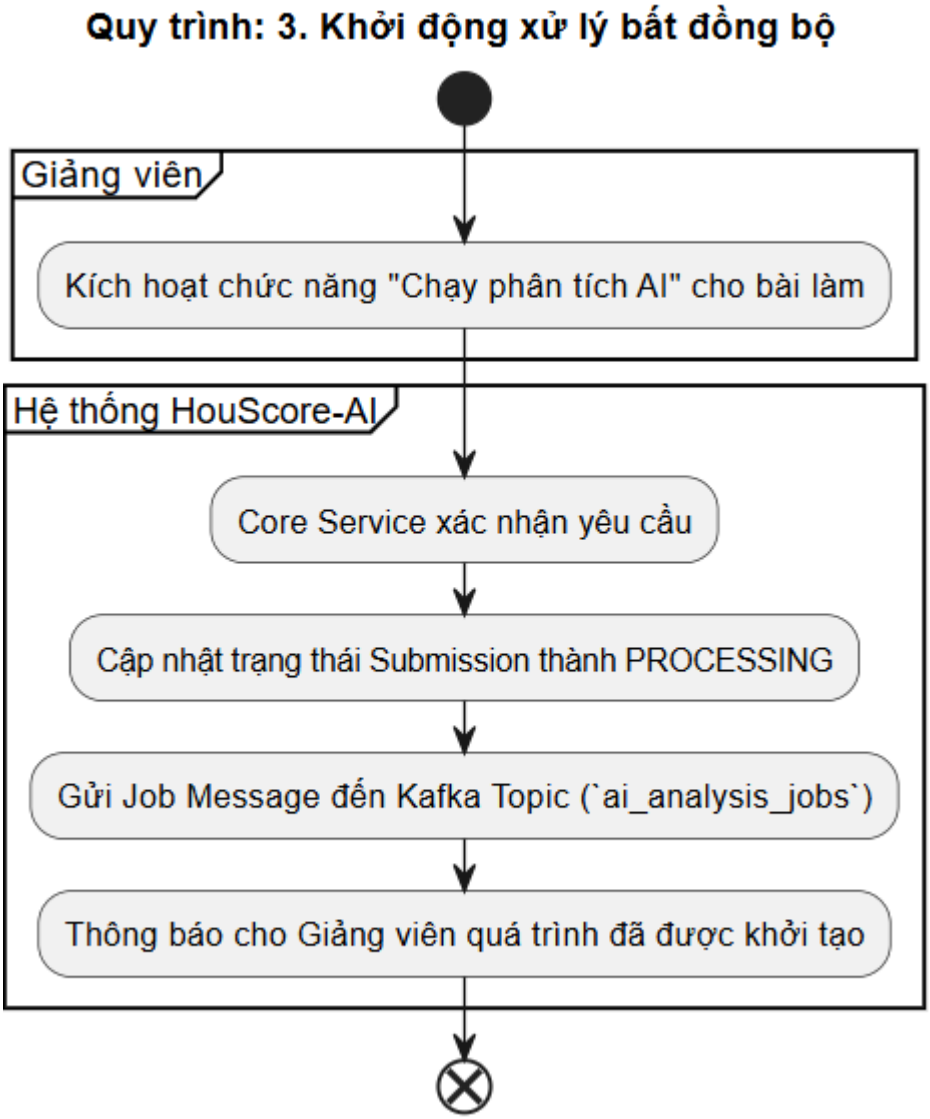
Mục tiêu của quy trình là chuyển yêu cầu phân tích tốn thời gian sang luồng xử lý bất đồng bộ, giải phóng tài nguyên của giao diện người dùng. Bao gồm các bước chính sau:

(1) Kích hoạt: giảng viên nhấn nút "chạy phân tích AI".

(2) Xác nhận: core service kiểm tra submission metadata và Rubric đã gán

(3) Gửi job: core service gửi một message nhẹ chứa submission_id, rubric_id,.. đến Kafka topic

(4) Phản hồi: cập nhật trạng thái submission thành PROCESSING và phản hồi giảng viên job đã được xếp hàng.



Hình 3. Quy trình khởi động xử lý bất đồng bộ

2.5.4. Quy trình thực thi AI core

Mục đích của quy trình là thực thi các mô hình AI để tạo ra điểm số, phát hiện đạo văn và phản hồi chi tiết. Các bước chính bao gồm:

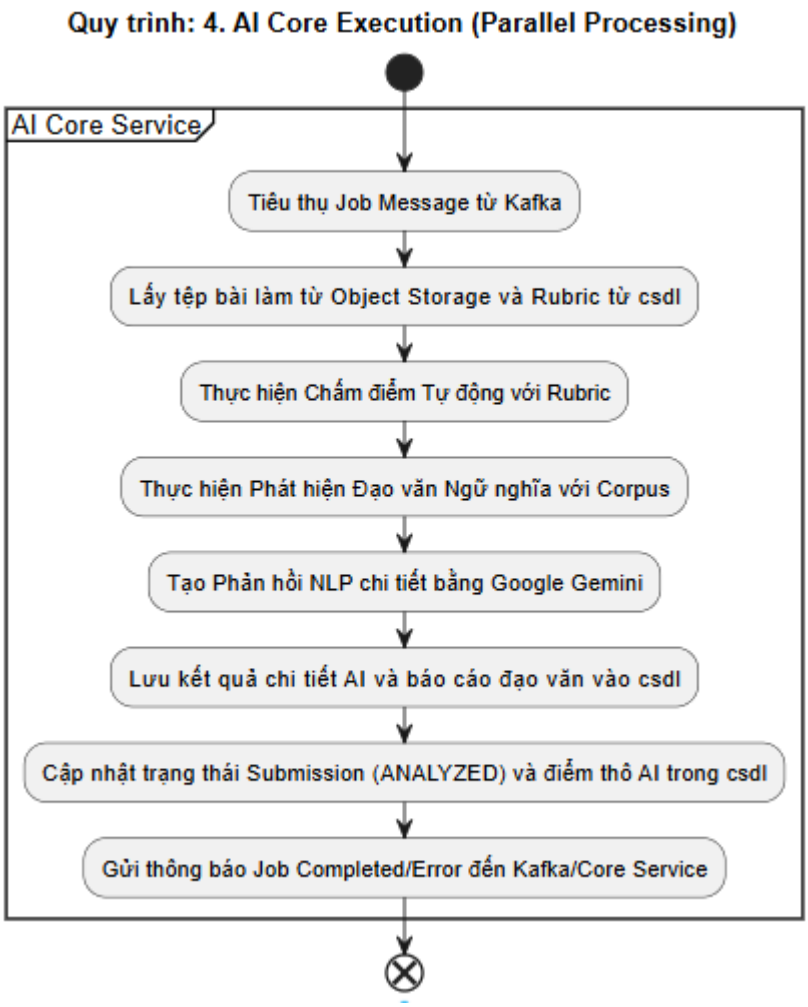
(1) Tiêu thụ job: AI core service tiêu thụ message từ Kafka.

(2) Xử lý: Thực hiện chấm điểm tự động và phát hiện đạo văn.

(3) Phản hồi NLP: gửi kết quả và văn bản làm đến Google Gemini để tạo các nhận xét có cấu trúc.

(4) Lưu trữ kết quả: Lưu báo cáo đạo văn và báo cáo chi tiết AI vào csdl.

(5) Cập nhật trạng thái status: cập nhật ai_score_raw và trạng thái ANALYZED.



Hình 4. Quy trình thực thi AI

2.5.5. Quy trình quản lý kết quả và hiệu chỉnh

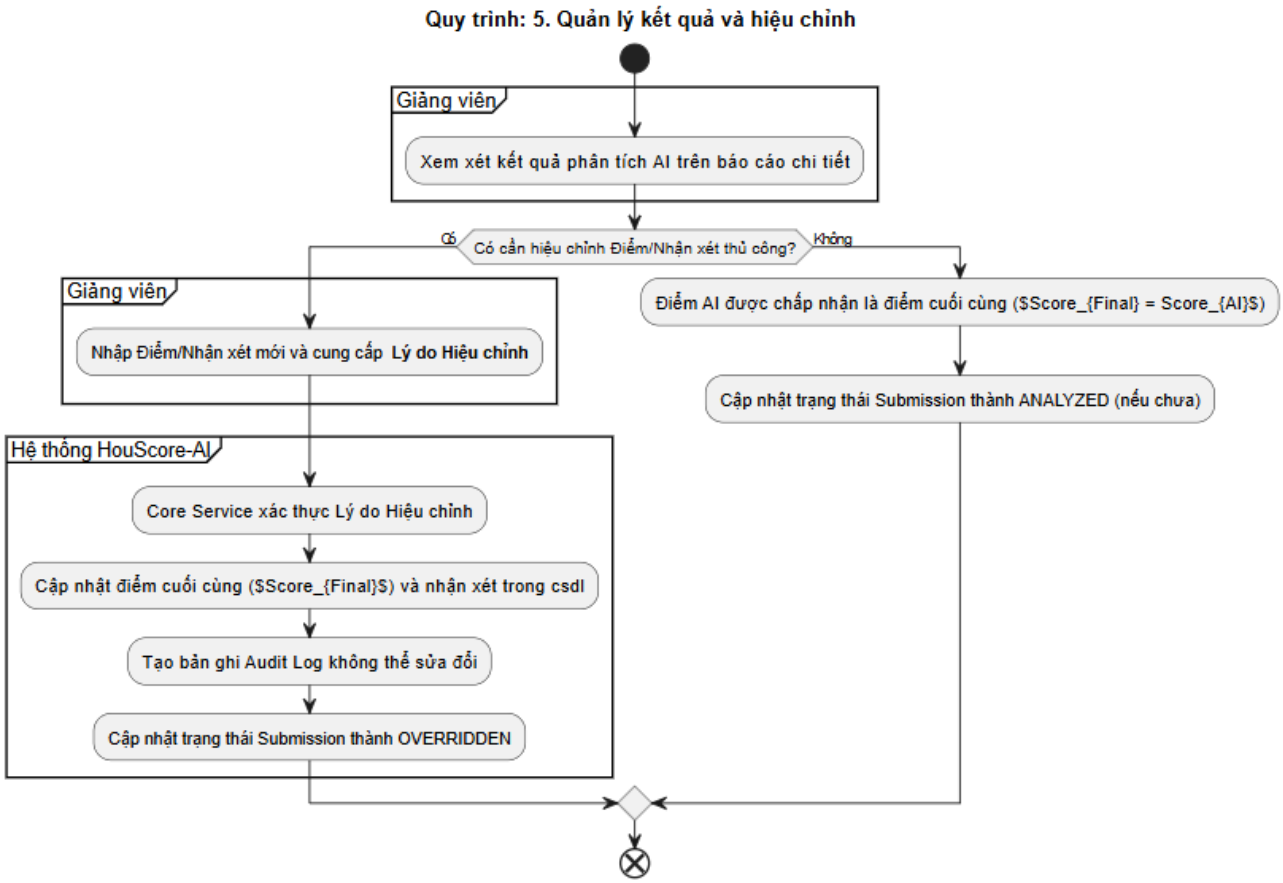
Mục tiêu của quy trình là cho phép giảng viên sử dụng kinh nghiệm chuyên môn để hiệu chỉnh kết quả tự động, duy trình quyền kiểm soát của con người. Các bước chính bao gồm:

(1) Xem xét: giảng viên xem báo cáo chi tiết

(2) Quyết định hiệu chỉnh: Nếu cần thay đổi điểm số, giản viên nhập Score final mới và bắt buộc phải nhập lý do hiệu chỉnh

(3) Ghi Audit: core service không chỉ cập nhật Score final mà còn tạo một bàn ghi Audit.log không thể sửa đổi trong csdl, ghi lại chi tiết sự thay đổi và lý do

(4) Cập nhật status: cập nhật trạng thái thành OVERRIDE nếu có hiệu chỉnh hoặc giữ nguyên ANALYZED



Hình 5. Quy trình quản lý kết quả và hiệu chỉnh

2.5.6. Quy trình phản hồi và phân phối

Mục đích là hoàn tất quy trình bằng cách công bố kết quả và đảm bảo điểm số được cập nhật chính xác trên tất cả các hệ thống liên quan. Các bước chính bao gồm:

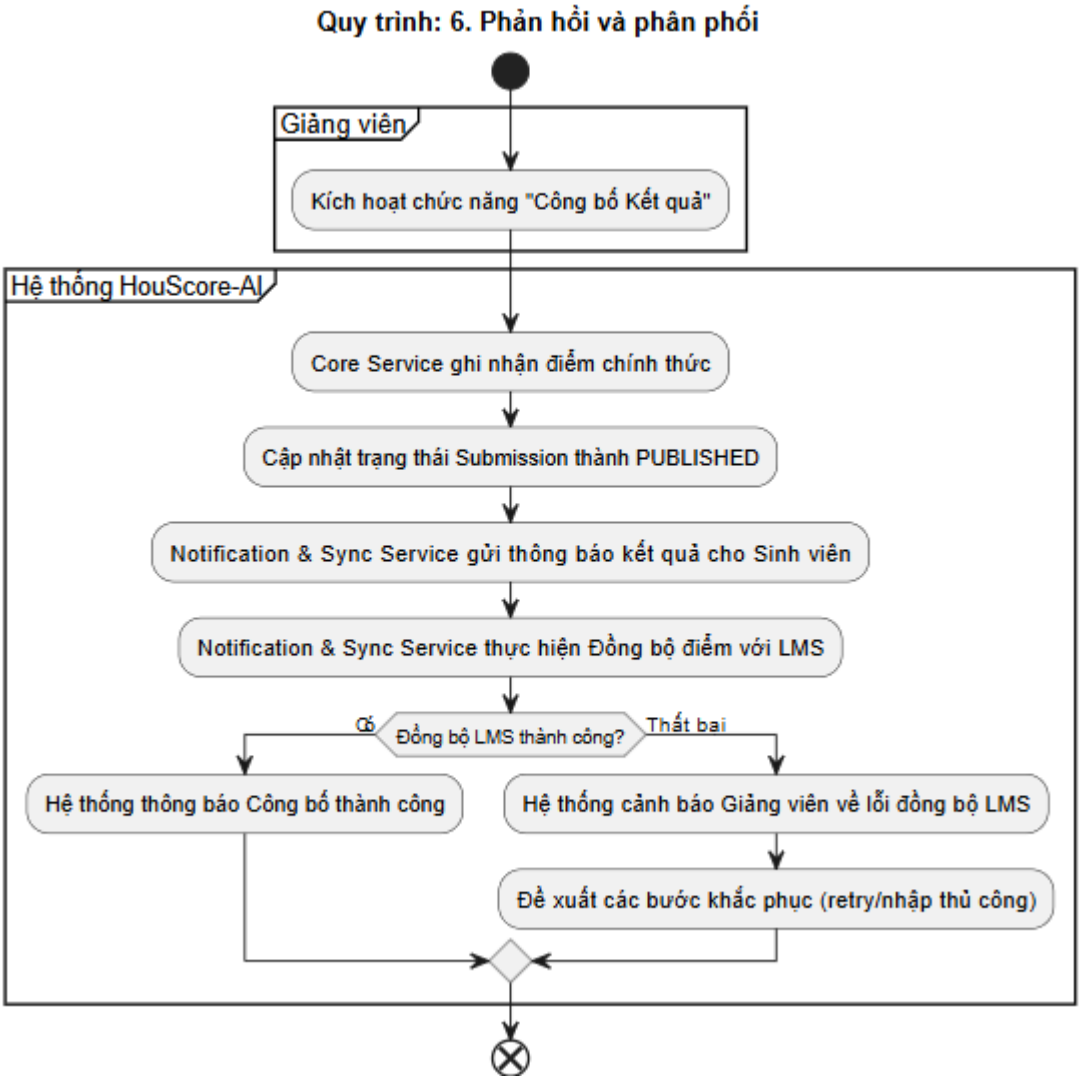
(1) Công bố: giảng viên kích hoạt công bố kết quả

(2) Ghi nhận: core service cập nhật trạng thái thành PUBLISHED

(3) Thông báo: notification và sync service gửi mail thông báo chứa liên kết an toán đến sinh viên

(4) Đồng bộ LMS: notification và sync service gọi API LMS để đẩy Score final tới LMS.

(5) Xử lý lỗi: nếu grade passback thất bại, hệ thống áp dụng cơ chế retry hoặc cảnh báo giảng viên để họ nhập điểm thủ công.



Hình 6. Quy trình phản hồi và phân phối

2.6. Đặc tả các Use Case chi tiết

2.6.1. Đặc tả Use Case Quản lý khóa học

Bảng 1. Đặc tả UC Quản lý khóa học

Use Case Name	Quản lý khóa học
Use Case ID	UC-09a
Use Case Description	Giảng viên tạo, chỉnh sửa thông tin Khóa học và đồng bộ danh sách Khóa học/Sinh viên từ Hệ thống LMS đã được tích hợp.
Actor	Giảng viên, Hệ thống LMS
Priority	Cao
Trigger	Giảng viên chọn chức năng Quản lý khóa học trên hệ thống.

Pre-Condition	Giảng viên đã đăng nhập thành công.
Post-Condition	Thông tin khóa học được cập nhật chính xác trong hệ thống HouScore-AI, và danh sách sinh viên được đồng bộ hóa.
Basic Flow	1. Giảng viên truy cập mục Quản lý khóa học. 2. Hệ thống hiển thị danh sách khóa học hiện có. 3. Giảng viên chọn tạo mới (nhập thông tin) hoặc đồng bộ từ LMS. 4. (Nếu Đồng bộ): Hệ thống gọi API LMS để kéo danh sách khóa học và Sinh viên mới/cập nhật. 5. Hệ thống lưu thông tin khóa học và roster (danh sách sinh viên) mới vào DB. 6. Hệ thống hiển thị thông báo thành công.
Alternative Flow	A. Sửa thông tin khóa học: 3. Giảng viên chọn một khóa học và chọn sửa. 4. Giảng viên cập nhật tên/mô tả và lưu. 5. Hệ thống cập nhật DB và thông báo thành công.
Exception Flow	E.1. Lỗi kết nối LMS: 4. Hệ thống không thể kết nối hoặc nhận lỗi từ API LMS. 5. Hệ thống thông báo "Đồng bộ thất bại, vui lòng kiểm tra lại cấu hình LMS (UC-10)".
Business Rule	BR-C1: Khóa học phải có tên và mã khóa học duy nhất. BR-C2: Quá trình đồng bộ LMS chỉ áp dụng cho các khóa học mà Giảng viên hiện tại có quyền quản lý.

2.6.2. Đặc tả Use Case Quản lý tài khoản giảng viên

Bảng 2. Đặc tả UC Quản lý tài khoản giảng viên

Use Case Name	Quản lý tài khoản giảng viên
Use Case ID	UC-09b

Use Case Description	Giảng viên thực hiện đăng nhập, cập nhật thông tin cá nhân (email, tên) và quản lý mật khẩu/yếu tố xác thực.
Actor	Giảng viên
Priority	Cao (Liên quan đến bảo mật)
Trigger	Giảng viên truy cập hệ thống (đăng nhập) hoặc truy cập mục cài đặt tài khoản (quản lý).
Pre-Condition	(Đăng nhập): Tài khoản đã được cấp. (quản lý): Giảng viên đã đăng nhập.
Post-Condition	Giảng viên được xác thực vào hệ thống hoặc thông tin tài khoản được cập nhật an toàn.
Basic Flow	1. Giảng viên nhập Tên đăng nhập/Email và Mật khẩu. 2. Hệ thống xác thực thông tin (hoặc thông qua SSO/OIDC). 3. Hệ thống cấp JWT và chuyển giảng viên đến trang chủ.
Alternative Flow	A. Cập nhật thông tin: 1. Giảng viên truy cập cài đặt. 2. Giảng viên chỉnh sửa tên, email. 3. Giảng viên chọn lưu. 4. Hệ thống cập nhật DB và thông báo thành công.
Exception Flow	E.1. Sai thông tin đăng nhập: 2. Hệ thống xác thực thất bại. 3. Hệ thống thông báo "Tên đăng nhập hoặc mật khẩu không đúng." E.2. Lỗi hệ thống khi cập nhật: 4. Hệ thống gặp lỗi khi ghi vào DB. 5. Hệ thống thông báo lỗi kỹ thuật.

Business Rule	BR-A1: Mật khẩu phải được băm (hashing) và không được lưu trữ dưới dạng văn bản thuần. BR-A2: Yêu cầu xác thực đa yếu tố (MFA) được bật cho tài khoản giảng viên (tính năng mở rộng).

2.6.3. Đặc tả Use Case Quản lý Rubric

Bảng 3. Đặc tả UC Quản lý Rubric

Use Case Name	Quản lý Rubric
Use Case ID	UC-06
Use Case Description	Giảng viên tạo, chỉnh sửa hoặc chọn các Rubric (tiêu chí) chấm điểm và thiết lập trọng số (Weight) cho từng tiêu chí trước khi chạy AI.
Actor	Giảng viên
Priority	Rất Cao (Cốt lõi của việc chấm điểm AI)
Trigger	Giảng viên cần định nghĩa tiêu chí chấm điểm cho một bài tập.
Pre-Condition	Giảng viên đã chọn khóa học và bài tập để cấu hình.
Post-Condition	Rubric được lưu trữ trong hệ thống với các tiêu chí và trọng số hợp lệ, sẵn sàng cho AI Core Service sử dụng.
Basic Flow	1. Giảng viên truy cập cấu hình Rubric. 2. Hệ thống hiển thị các Rubric đã lưu. 3. Giảng viên chọn tạo Rubric mới. 4. Giảng viên nhập tên Rubric, thêm các tTiêu chí (`Criterion`) và đặt trọng số (`Weight`) cho từng tiêu chí. 5. Giảng viên chọn lưu. 6. Hệ thống lưu trữ cấu trúc Rubric (bao gồm tổng trọng số = 100%). 7. Hệ thống thông báo thành công.
Alternative Flow	A. Sửa Rubric: 3. Giảng viên chọn một Rubric hiện có và chọn sửa.

	4. Giảng viên thay đổi tiêu chí hoặc trọng số (FR-2.1). 5. Hệ thống kiểm tra tính hợp lệ của tổng trọng số trước khi lưu.
Exception Flow	E.1. Trọng số không hợp lệ: 5. Hệ thống phát hiện tổng trọng số ≠100%. 6. Hệ thống yêu cầu giảng viên điều chỉnh lại trọng số. E.2. Tiêu chí trùng lặp: 5. Hệ thống cảnh báo tiêu chí bị trùng lặp.
Business Rule	BR-R1 (FR-2.1): Trọng số của tất cả các tiêu chí trong một Rubric phải cộng lại bằng 100%. BR-R2: Mỗi Bài tập phải được gán một Rubric hợp lệ trước khi chạy phân tích AI.

2.6.4. Đặc tả Use Case Quản lý kho dữ liệu – Corpus

Bảng 4. Đặc tả UC Quản lý kho dữ liệu

Use Case Name	Quản lý kho dữ liệu (Corpus)
Use Case ID	UC-07
Use Case Description	Giảng viên tải lên các tài liệu nội bộ (sách, bài giảng) để xây dựng kho dữ liệu so sánh, phục vụ cho việc phát hiện đạo văn ngữ nghĩa (FR-2.2).
Actor	Giảng viên
Priority	Cao (Cần thiết cho phát hiện đạo văn)
Trigger	Giảng viên muốn bổ sung nguồn dữ liệu để so sánh đạo văn.
Pre-Condition	Giảng viên đã đăng nhập thành công.
Post-Condition	Tài liệu mới được lưu vào Object Storage và được lập chỉ mục Vector Embedding để phục vụ so sánh ngữ nghĩa.
Basic Flow	1. Giảng viên truy cập mục quản lý kho dữ liệu. 2. Giảng viên chọn tải lên tài liệu Corpus.

	3. Giảng viên chọn tệp (PDF, DOCX). 4. Hệ thống tải tệp lên Object Storage. 5. Hệ thống gửi một Job bất đồng bộ tới AI Core Service. 6. AI Core Service chuyển đổi tài liệu thành Vector (dùng Google Gemma) và lưu vào Vector DB (FR-2.2). 7. Hệ thống thông báo cho Giảng viên "Tài liệu đang được xử lý/đã sẵn sàng sử dụng".
Alternative Flow	A. Xóa Tài liệu: 3. Giảng viên chọn một tài liệu hiện có và chọn Xóa. 4. Hệ thống xóa tệp khỏi Object Storage và hủy chỉ mục khỏi Vector DB. 5. Hệ thống thông báo thành công.
Exception Flow	E.1. Lỗi định dạng tệp: 4. Hệ thống từ chối tệp không hợp lệ. 5. Hệ thống hiển thị cảnh báo về định dạng tệp được hỗ trợ. E.2. Lỗi Vectorization: 6. AI Core Service gặp lỗi khi chuyển đổi. 7. Hệ thống đánh dấu trạng thái tài liệu là "Lỗi xử lý".
Business Rule	BR-CP1 (FR-2.2): Tài liệu Corpus phải được lập chỉ mục dưới dạng Vector Embedding trước khi có thể được sử dụng trong quá trình Phát hiện Đạo văn. BR-CP2: Kích thước tệp tối đa cho mỗi lần tải lên là 50MB.

2.6.5. Đặc tả Use Case Tải bài làm – Submission

Bảng 5. Đặc tả UC Tải bài làm Submission

Use Case Name	Tải bài làm (Submission)
Use Case ID	**UC-01**
Use Case Description	Giảng viên tải lên các bài làm của sinh viên để bắt đầu quy trình chấm điểm và phân tích.

Actor	Giảng viên
Priority	Cao
Trigger	Giảng viên đã thu thập bài làm và sẵn sàng đưa vào hệ thống.
Pre-Condition	Khóa học và bài tập đã được tạo trên hệ thống.
Post-Condition	Tệp bài làm được lưu trữ an toàn, và metadata của submission được tạo, sẵn sàng để chạy phân tích AI.
Basic Flow	1. Giảng viên chọn khóa học và bài tập. 2. Giảng viên tải lên tệp bài làm (hỗ trợ .docx, .pdf, .txt). 3. Hệ thống (Data Ingestion) tiếp nhận tệp (FR-1.1), kiểm tra virus, và lưu vào Object Storage. 4. Hệ thống (Core Service) tạo bản ghi Metadata `Submission` trong csdl (Status: `UPLOADED`). 5. Hệ thống thông báo "Tải lên thành công".
Alternative Flow	A. Tải lên hàng loạt: 2. Giảng viên tải lên tệp ZIP/RAR chứa nhiều bài làm, mỗi tên tệp khớp với ID sinh viên. 3. Hệ thống tự động giải nén và xử lý từng tệp theo luồng chính.
Exception Flow	E.1. Tệp không được hỗ trợ: 3. Hệ thống từ chối và thông báo định dạng tệp không hợp lệ. E.2. Lỗi lưu trữ: 4. Hệ thống gặp lỗi khi ghi vào Object Storage. 5. Hệ thống thông báo lỗi và yêu cầu thử lại.
Business Rule	BR-S1 (FR-1.1): Hệ thống chỉ chấp nhận tệp DOCX, PDF, TXT. BR-S2: Mỗi tệp phải được gán ID Sinh viên và ID Bài tập duy nhất.

2.6.6. Đặc tả Use Case Chạy phân tích AI

Bảng 6. Đặc tả UC Chạy phân tích AI

Use Case Name	Chạy phân tích AI
Use Case ID	**UC-02**
Use Case Description	Khởi động quá trình phân tích bất đồng bộ, bao gồm chấm điểm tự động (FR-1.2) và phát hiện đạo văn ngữ nghĩa (FR-1.3).
Actor	Giảng viên
Priority	Rất cao (Cốt lõi chức năng)
Trigger	Giảng viên kích hoạt nút "Chạy AI" sau khi đã tải bài làm và cấu hình Rubric.
Pre-Condition	Tệp bài làm đã được tải lên (UC-01) và Rubric đã được gán (UC-06).
Post-Condition	Metadata Submission được cập nhật trạng thái (`PROCESSING`), và các kết quả phân tích thô của AI được lưu vào csdl.
Basic Flow	1. Giảng viên chọn bài làm/lô bài làm và kích hoạt "Chạy AI". 2. Hệ thống (Core Service) cập nhật `Submission Status` thành `PROCESSING`. 3. Hệ thống gửi Job Message tới Kafka Topic (`ai_analysis_jobs`). 4. AI Core Service tiêu thụ message, thực hiện FR-1.2 (Chấm điểm ML) và FR-1.3 (Đạo văn Semantic Search). 5. AI Core Service lưu báo cáo chi tiết vào csdl. 6. AI Core Service gửi signal hoàn thành. 7. Core Service cập nhật `Submission Status` thành `ANALYZED`.
Alternative Flow	A. Chạy lại Phân tích: 1. Giảng viên chọn bài làm đã hoàn thành và chạy lại. 2. Hệ thống ghi đè các kết quả phân tích cũ bằng kết quả mới (sau khi xác nhận của Giảng viên).

Exception Flow	E.1. Lỗi xử lý AI: 4. AI Core Service gặp ngoại lệ. 5. AI Core Service gửi signal lỗi. 6. Core Service cập nhật `Submission Status` thành `ERROR` và ghi log chi tiết. 7. Giảng viên nhận thông báo lỗi.
Business Rule	BR-AI1 (NFR-CON-3): Quá trình phân tích phải được thực hiện bất đồng bộ (Async) để không làm ảnh hưởng đến hiệu suất hệ thống chính. BR-AI2: Phải sử dụng Kho dữ liệu Corpus nếu có.

2.6.7. Đặc tả Use Case Hiệu chỉnh điểm và nhận xét

Bảng 7. Đặc tả UC Hiệu chỉnh điểm và nhận xét

Use Case Name	Hiệu chỉnh điểm và nhận xét
Use Case ID	**UC-08**
Use Case Description	Giảng viên xem xét kết quả AI và có quyền thủ công điều chỉnh điểm số cuối cùng (ScoreFinal) hoặc nhận xét tự động trước khi chính thức công bố.
Actor	Giảng viên
Priority	Cao (Liên quan đến sự tin cậy của con người)
Trigger	Giảng viên xem kết quả phân tích AI (`ANALYZED`) và nhận thấy cần thay đổi.
Pre-Condition	Bài làm phải có trạng thái `ANALYZED` (đã có kết quả AI).
Post-Condition	Điểm số và nhận xét được cập nhật, và một bản ghi Audit Trail được tạo.
Basic Flow	1. Giảng viên truy cập báo cáo bài làm. 2. Giảng viên chỉnh sửa ScoreFinal và/hoặc nội dung nhận xét. 3. Giảng viên phải nhập Lý do Hiệu chỉnh (`audit_reason`). 4. Giảng viên chọn Lưu. 5. Hệ thống cập nhật các giá trị mới. 6. Hệ thống tạo bản ghi AuditLog (FR-3.2) chi tiết trong csdl.

	7. Hệ thống thông báo thành công.
Alternative Flow	A. Chỉnh sửa nhận xét AI: 2. Giảng viên chỉ chỉnh sửa phần nhận xét (lưu ý: phần điểm AI ScoreAI gốc vẫn được giữ nguyên cho mục đích đối chiếu).
Exception Flow	E.1. Thiếu lý do hiệu chỉnh: 3. Hệ thống ngăn giảng viên lưu lại nếu trường `audit_reason` bị trống. 4. Hệ thống hiển thị cảnh báo.
Business Rule	BR-O1 (FR-3.2): Mọi sự thay đổi điểm số hoặc trạng thái đạo văn thủ công đều bắt buộc phải ghi lại Lý do Hiệu chỉnh và tạo bản ghi Audit Trail không thể sửa đổi. BR-O2: Hệ thống phải lưu trữ cả ScoreAI (Điểm gốc của AI) và ScoreFinal (Điểm cuối cùng sau hiệu chỉnh).

2.6.8. Đặc tả Use Case Công bố kết quả

Bảng 8. Đặc tả UC Công bố kết quả

Use Case Name	Công bố kết quả
Use Case ID	**UC-04**
Use Case Description	Giảng viên chính thức công bố điểm số cuối cùng và kích hoạt cơ chế gửi thông báo/đồng bộ điểm với LMS.
Actor	Giảng viên, Hệ thống LMS
Priority	Cao
Trigger	Giảng viên đã hoàn thành việc xem xét và hiệu chỉnh (nếu cần) kết quả.
Pre-Condition	Trạng thái bài làm là `ANALYZED` hoặc `OVERRIDDEN`.
Post-Condition	Điểm cuối cùng được ghi nhận (FR-4.1), Sinh viên nhận được thông báo, và điểm số được đẩy sang LMS (FR-5.1).
Basic Flow	1. Giảng viên chọn Bài tập/Khóa học và kích hoạt "Công bố kết quả". 2. Hệ thống (Core Service) cập nhật trạng thái thành `PUBLISHED` (FR-4.1). 3. Hệ thống gọi Notification Service. 4. Notification Service thực hiện FR-4.2 (Gửi Email/Thông báo LMS).

	5. Notification Service thực hiện FR-5.1 (Grade Passback) bằng cách gọi API LMS. 6. Hệ thống thông báo Giảng viên quá trình công bố đã hoàn tất.
Alternative Flow	A. Công bố theo từng đợt: 1. Giảng viên chỉ chọn một nhóm bài làm để công bố trước. 2. Luồng tiếp tục như Basic Flow, chỉ áp dụng cho nhóm đã chọn.
Exception Flow	E.1. Lỗi Grade Passback (FR-5.1): 5. Hệ thống nhận mã lỗi từ API LMS. 6. Notification Service thực hiện cơ chế Retry (thử lại) với Exponential Backoff. 7. Nếu Retry thất bại: 8. Hệ thống cảnh báo Giảng viên và yêu cầu nhập điểm thủ công vào LMS.
Business Rule	BR-P1 (FR-5.1): Grade Passback là bắt buộc nếu khóa học đã được cấu hình tích hợp LMS. BR-P2 (FR-4.1): Bài làm phải ở trạng thái `PUBLISHED` mới cho phép Sinh viên xem chi tiết.

2.6.9. Đặc tả Use Case Xem kết quả chi tiết bài làm

Bảng 9. Đặc tả UC Xem kết quả chi tiết bài làm

Use Case Name	Xem kết quả chi tiết bài làm
Use Case ID	**UC-05**
Use Case Description	Sinh viên xem điểm số cuối cùng, phân tích Rubric, nhận xét tự động (FR-4.3) và báo cáo đạo văn ngữ nghĩa (nếu có).
Actor	Sinh viên
Priority	Cao
Trigger	Sinh viên nhận được thông báo (Email/LMS) và truy cập liên kết kết quả.
Pre-Condition	Bài làm phải có trạng thái `PUBLISHED` (UC-04).
Post-Condition	Sinh viên đã xem xong kết quả.

Basic Flow	1. Sinh viên nhấp vào liên kết an toàn (Secure One-Time Link). 2. Hệ thống xác thực tính hợp lệ của Token/Liên kết. 3. Hệ thống truy vấn Metadata Submission và báo cáo chi tiết (`ai_report_json` từ csdl). 4. Hệ thống hiển thị giao diện Annotated View (bao gồm ScoreFinal, phân tích Rubric, Phản hồi NLP chi tiết - FR-4.3). 5. Hệ thống hiển thị báo cáo đạo văn ngữ nghĩa nếu mức độ trùng lặp cao.
Alternative Flow	A. Truy cập qua LMS: 1. Sinh viên truy cập trực tiếp qua LMS Gradebook. 2. Hệ thống LMS chuyển hướng an toàn đến HouScore-AI bằng LTI/OAuth Token. 3. Luồng tiếp tục như Basic Flow từ bước hiển thị.
Exception Flow	E.1. Liên kết hết hạn/Không hợp lệ: 2. Hệ thống từ chối truy cập. 3. Hệ thống yêu cầu Sinh viên liên hệ giảng viên để được cấp lại quyền truy cập.
Business Rule	BR-V1: Hệ thống bắt buộc phải sử dụng liên kết an toàn hoặc token xác thực để bảo vệ quyền riêng tư. BR-V2 (FR-4.3): Phản hồi NLP phải được hiển thị dưới dạng có cấu trúc (ví dụ: JSON Schema đã định nghĩa).

2.6.10. Đặc tả Use Case Cấu hình tích hợp LMS

Bảng 10. Đặc tả UC Cấu hình tích hợp LMS

Use Case Name	Cấu hình tích hợp LMS
Use Case ID	**UC-10**
Use Case Description	Giảng viên thiết lập kết nối (API Key, OAuth Credentials) với hệ thống Quản lý học tập bên ngoài (LMS) để kích hoạt đồng bộ hóa dữ liệu.

Actor	Giảng viên, Hệ thống LMS
Priority	Cao
Trigger	Giảng viên muốn liên kết khóa học trên HouScore-AI với khóa học trên LMS.
Pre-Condition	Giảng viên phải có quyền quản trị/Developer Key trên LMS.
Post-Condition	Các thông số kết nối API LMS được lưu trữ an toàn trong hệ thống HouScore-AI.
Basic Flow	1. Giảng viên truy cập cài đặt tích hợp LMS. 2. Giảng viên chọn LMS cần kết nối (ví dụ: Canvas, Moodle). 3. Giảng viên nhập các thông số OAuth Client ID/Secret hoặc API Key. 4. Giảng viên chọn kiểm tra kết nối. 5. Hệ thống thực hiện một API Call thử nghiệm tới LMS. 6. Nếu thành công: 7. Hệ thống mã hóa và lưu trữ các thông số kết nối trong Secret Manager. 8. Hệ thống thông báo "Kết nối LMS thành công".
Alternative Flow	A. Cập nhật thông số: 3. Giảng viên thay đổi Client Secret hoặc API Key. 4. Luồng tiếp tục như Basic Flow từ bước kiểm tra kết nối.
Exception Flow	E.1. Kiểm tra kết nối thất bại: 5. Hệ thống nhận mã lỗi xác thực/kết nối từ LMS. 6. Hệ thống hiển thị lỗi và yêu cầu giảng viên kiểm tra lại thông số (Client ID, Secret/Key).
Business Rule	BR-L1: Các thông số API/OAuth (Client ID/Secret) phải được lưu trữ mã hóa trong Secret Manager, KHÔNG được lưu trong DB Transactional dưới dạng plaintext.

	BR-L2: Hệ thống phải có cơ chế Test Connection trước khi lưu trữ thông số.

3. Các yêu cầu phi chức năng

Các yêu cầu phi chức năng mô tả các tiêu chí để đánh giá hoạt động của hệ thống, không liên quan đến chức năng cụ thể.

3.1. Khả năng thực thi – Performance

Bảng 11. NFR Khả năng thực thi

ID	Yêu cầu	Loại	Mô tả chi tiết
NFR-PER-1	Thời gian phản hồi API	Độ trễ	Các API đồng bộ (Đăng nhập, Quản lý Rubric) phải có thời gian phản hồi dưới 500 ms (median).
NFR-PER-2	Thời gian Xử lý AI	Throughput	Hệ thống phải có khả năng xử lý bài làm với tốc độ tối thiểu 0.05 bài/giây (tức là 1 bài/20 giây) trong điều kiện tải trung bình (bao gồm cả chấm điểm và phát hiện đạo văn).
NFR-PER-3	Khả năng mở rộng (Scaling)	Khả năng mở rộng	AI Core Service phải hỗ trợ Horizontal Scaling (HPA) dựa trên độ trễ của Kafka Consumer Group để xử lý các đợt tải bài lớn.

3.2. Bảo mật – Security

Bảng 12. NFR Bảo mật

ID	Yêu cầu	Loại	Mô tả chi tiết
NFR-SEC-1	Xác thực an toàn	Truy cập	Phải hỗ trợ cơ chế xác thực an toàn thông qua OAuth 2.0 / LTI hoặc sử dụng BCrypt hashing cho mật khẩu truyền thống.
NFR-SEC-2	Mã hóa dữ liệu nhạy cảm	Bảo vệ dữ liệu	Các thông số kết nối LMS (API Key/Secret) và các tài liệu Corpus phải được lưu trữ mã hóa trong Secret Manager (Vault/AWS/GCP) và Object Storage (Server-Side Encryption).

NFR-SEC-3	Kiểm soát truy cập	Ủy quyền	Phải thực thi RBAC (Role-Based Access Control): Sinh viên chỉ xem được kết quả của chính mình; Giảng viên chỉ quản lý được các khóa học của họ.
NFR-SEC-4	Bảo mật truyền tải	Mã hóa	Tất cả giao tiếp (Client ↔ Server và Service ↔ Service) phải sử dụng HTTPS/TLS (ưu tiên mTLS cho giao tiếp nội bộ Microservices).

3.3. Khả năng vận hành

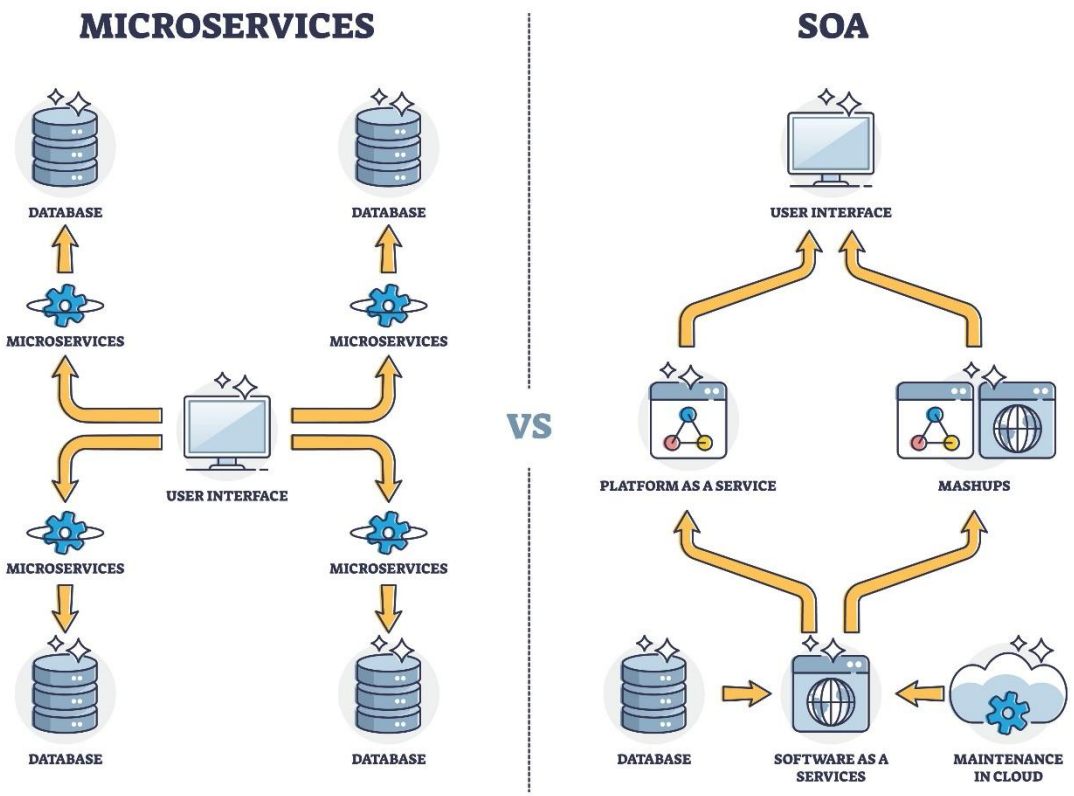
Bảng 13. NFR Khả năng vận hành

ID	Yêu cầu	Loại	Mô tả chi tiết
NFR-OPE-1	Giám sát tập trung	Giám sát	Hệ thống phải có cơ chế thu thập Metrics, Traces và Logs từ tất cả Microservices vào một hệ thống tập trung (Prometheus/Grafana/ELK).
NFR-OPE-2	Tính sẵn sàng (Uptime)	Sẵn sàng	Hệ thống phải đạt 99.9% Uptime (không tính thời gian bảo trì định kỳ), được hỗ trợ bởi kiến trúc K8s.
NFR-OPE-3	Khả năng phục hồi	Chịu lỗi	AI Core Service phải có cơ chế Retry Policy và xử lý lỗi khi kết nối LMS bị gián đoạn (FR-5.1) và khi xử lý Job Kafka bị thất bại.

4. Thiết kế hệ thống

4.1. Kiến trúc tổng thể

Hệ thống HouScore-AI được thiết kế dựa trên kiến trúc Microservices và Polyglot Persistence để tối tưu hóa hiệu suất cho các tác vụ xử lý ngôn ngữ tự nhiên (NLP) tốn tài nguyên. Hệ thống được triển khai trên nền tảng Kubernetes (K8s) để quản lý tài nguyên tính toán chuyên biệt (GPU/TPU) và đảm bảo tính sẵn sàng cao.



Hình 7. Mô hình Microservices

Hệ thống được phân tách thành các dịch vụ độc lập giao tiếp qua API Gateway cho các yêu cầu đồng bộ và Apache Kafka cho các job bất đồng bộ.

Bảng 14. Bảng mô tả kiến trúc tổng thể HouScore-AI

Microservice	Trách nhiệm chính	Công nghệ khuyến nghị
API Gateway	Xác thực (JWT), Ủy quyền, Định tuyến (Routing), Giới hạn tỷ lệ (Rate Limiting).	Kong / Nginx + Auth Layer
Core Service	Logic nghiệp vụ: CRUD Rubric (UC-06), Quản lý Metadata Submission (UC-01), Audit Trail (FR-3.2), Khởi tạo Job Kafka (UC-02).	Laravel (PHP) / Spring Boot

AI Core Service	Compute-Intensive: Thực thi mô hình AI (Gemma/Gemini), FR-1.2 (Chấm điểm), FR-1.3 (Đạo văn), Vectorization.	Python (PyTorch/TensorFlow)
Data Ingestion	Xử lý Tệp: Tiếp nhận, Tiền xử lý (FR-1.1), Lưu trữ tệp thô vào Object Storage.	Go / Python (nhanh, hiệu quả I/O)
Notification & Sync	Liên thông: FR-4.2 (Gửi Email/Thông báo), FR-5.1 (Grade Passback đến LMS), Xử lý Retry Policy.	Go / Node.js

4.2. Thiết kế lớp dữ liệu

Hệ thống sử dụng chiến lược Polyglot Persistence, chọn loại cơ sở dữ liệu phù hợp nhất cho từng nhu cầu dữ liệu, được mô tả trong bảng dưới đây:

Bảng 15. Thiết kế lớp dữ liệu

Lớp dữ liệu	Công nghệ	Dịch vụ sở hữu	Mục đích lưu trữ
Transactional & Core Data	PostgreSQL	Core Service	Lưu trữ dữ liệu cấu trúc: User, Course, Rubric, Submission Metadata (Status, Final Score), và AuditLog (FR-3.2). Đảm bảo tính ACID.
Flexible & Report Data	MongoDB	AI Core Service	Lưu trữ dữ liệu bán cấu trúc, không đồng nhất: Báo cáo chi tiết AI (`ai_report_json` - FR-4.3), Báo cáo đạo văn (FR-1.3).

Vector Index	PgVector / Pinecone	AI Core Service	Lưu trữ Vector Embeddings của Corpus (UC-07) và các bài nộp. Phục vụ truy vấn Semantic Search tốc độ cao.
Blob Storage	AWS S3 / GCS	Data Ingestion	Lưu trữ tệp bài làm thô (DOCX/PDF - UC-01) và tài liệu Corpus dung lượng lớn.

4.3. Thiết kế luồng xử lý bất đồng bộ

Quy trình chấm điểm AI được thiết kế bất đồng bộ hoàn toàn thông qua Apache Kafka để đảm bảo hệ thống có thể xử lý hàng trăm bài làm đồng thời mà không bị nghẽn.

4.3.1. Cơ chế Kafka

- Job Producer (Core Service): Sau khi giảng viên kích hoạt "Chạy AI" (UC-02), Core Service tạo một message tối thiểu (`submission_id`, `rubric_id`, `file_path_s3`) và gửi lên topic `ai_analysis_jobs`.
- Job Consumer (AI Core Service): Các Pod AI Core (chạy trên K8s GPU Nodes) liên tục tiêu thụ message từ topic này.
- Status Updates: AI Core Service gửi các thông báo trạng thái job qua một topic khác (`ai_status_updates`) hoặc cập nhật trực tiếp DB sau khi hoàn thành.

4.3.2. Triển khai Kubernestes (K8s) và Auto-Scaling

- AI Core Deployment: AI Core Service được triển khai trên Node Pools K8s chuyên biệt với GPU Acceleration để xử lý các mô hình Deep Learning (Gemma, Gemini) hiệu quả.
- Horizontal Pod Autoscaler (HPA): K8s HPA được cấu hình để theo dõi độ trễ (Lag) của Consumer Group trên Kafka.
  - ✓ *Điều kiện mở rộng:* Nếu Consumer Lag vượt quá ngưỡng (ví dụ: > 100 jobs chưa xử lý), HPA sẽ tự động tăng số lượng Pod AI Core để tăng tốc độ tiêu thụ.
  - ✓ *Điều kiện thu hẹp:* Khi Lag giảm, HPA sẽ giảm số lượng Pod để tối ưu chi phí tài nguyên GPU.

4.4. Thiết kế tích hợp và bảo mật

4.4.1. Tích hợp hệ thống LMS

Bảng 16. Tích hợp hệ thống LMS

Giao thức	Mục đích	Dịch vụ liên quan	Chi tiết kỹ thuật
OAuth 2.0 / LTI	Xác thực và ủy quyền giảng viên để truy cập LMS.	Core Service	Được sử dụng trong quá trình cấu hình (UC-10) để giảng viên cấp quyền.
API RESTful	Đồng bộ khóa học (GET), Grade Passback (PUT/POST).	Notification & Sync	Sử dụng LMS API Keys hoặc OAuth Tokens đã lưu an toàn để thực hiện các cuộc gọi API.
Retry Policy	Đảm bảo truyền tải điểm số thành công.	Notification & Sync	Áp dụng cơ chế Exponential Backoff cho các lỗi API LMS (FR-4.2, E.1).

4.4.2. Thiết kế bảo mật

- Xác thực và ủy quyền:
  - ✓ Sử dụng JWT (JSON Web Token) để xác thực phiên làm việc.
  - ✓ Sử dụng OAuth 2.0/OIDC cho SSO và tích hợp với LMS (UC-09b, UC-10).
- Bảo mật Truyền tải (In Transit): Toàn bộ giao tiếp giữa các Microservices phải được mã hóa bằng mTLS (Mutual TLS). Giao tiếp ngoài (API Gateway) sử dụng HTTPS.
- Quản lý Bí mật (NFR-SEC-2): Các khóa API nhạy cảm (Google Gemini, LMS Secret, Database Credentials) phải được lưu trữ trong một Secret Manager chuyên dụng (ví dụ: HashiCorp Vault hoặc K8s Secrets được mã hóa).
- Audit Trail (FR-3.2):
  - ✓ Thực thi quy tắc Business Rule BR-O1 (mọi hành động hiệu chỉnh đều phải ghi log).
  - ✓ Lưu trữ các bản ghi AuditLog trong bảng chuyên biệt trong DB để đảm bảo tính toàn vẹn (ACID) và không thể thay đổi. Bản ghi bao gồm `user_id`, `action_type`, `submission_id`, `old_value`, `new_value`, và trường **`audit_reason`** bắt buộc.

Thiết kế này đảm bảo rằng HouScore-AI không chỉ đáp ứng các yêu cầu chức năng cốt lõi mà còn đáp ứng nghiêm ngặt các yêu cầu về hiệu suất, khả năng mở rộng và bảo mật.

4.5. Thiết kế AI Core

AI Core Service là trái tim của hệ thống, nơi thực hiện các tính toán phức tạp nhất. Thiết kế này tập trung vào hiệu quả và khả năng mở rộng của các mô hình AI.

4.5.1. Kiến trúc Pipeline xử lý AI

Khi nhận một Job từ Kafka, AI Core Service sẽ thực hiện pipeline sau:

Đoạn mã

```
      graph TD
          A[Kafka Message (submission_id, file_path)] --> B(Data Fetcher:
Lấy file từ S3)
          B --> C(Pre-processing: Trích xuất Text, Làm sạch - FR-1.1)
          C --> D{Text Splitting: Chia thành đoạn/câu}
          D --> E[Vectorization Engine: Google Gemma Embeddings]
          E --> F{FR-1.2: AI Scoring Model}
          F --> G[Score Aggregation & Rubric Application]
          G --> H{FR-1.3: Semantic Plagiarism Detection}
          H --> I[Plagiarism Report Generation]
          I --> J{FR-4.3: NLP Feedback Engine: Google Gemini}
          J --> K[Detailed AI Report JSON]
          K --> L(Data Persistence: Lưu MongoDB, Cập nhật PostgreSQL)
          L --> M[Kafka Status Update (Job Completed/Error)]
```

- FR-1.1: Tiếp nhận và tiền xử lý File
  - ✓ **Trách nhiệm:** Data Ingestion Service.
  - ✓ **Chi tiết:** Trích xuất văn bản từ DOCX/PDF/TXT, loại bỏ định dạng, tiêu đề/chân trang, ký tự đặc biệt. Chuyển đổi sang định dạng văn bản thuần túy (plain text) để AI dễ dàng xử lý.
- FR-1.2: Chấm điểm tự động (Semantic Scoring)
  - ✓ **Mô hình: Transformer Encoder (Custom/Fine-tuned BERT-like)** với Regression Head.
  - ✓ **Đầu vào:** Vector Embeddings (từ Google Gemma) của các đoạn văn bản trong bài làm, kết hợp với metadata Rubric (tiêu chí, trọng số).
  - ✓ **Đầu ra:** Điểm số dự đoán cho từng tiêu chí Rubric.
  - ✓ **Logic:** ScoreAI=∑i=1n(Predicted Scorei×Rubric Weighti).

- FR-1.3: Phát hiện đạo văn ngữ nghĩa (Semantic Plagiarism)
  - ✓ **Engine:** Sử dụng **Vector DB (PgVector/Pinecone)**.
  - ✓ **Quá trình:**
  - ✓ **Vectorization:** Các đoạn văn bản từ bài làm được chuyển thành Vector Embeddings (Gemma).
  - ✓ **Semantic Search:** Thực hiện truy vấn **Approximate Nearest Neighbor (ANN)** trên Vector DB để so sánh với:
    - Kho Corpus nội bộ (UC-07).
    - Các bài làm khác trong cùng khóa học (Cross-submission comparison).
  - ✓ **Similarity Threshold:** Xác định ngưỡng Cosine Similarity (`T_plagiarism`, ví dụ >0.85) để đánh dấu các đoạn văn bản có khả năng đạo văn cao.
  - ✓ **Báo cáo:** Tạo báo cáo JSON chi tiết các đoạn trùng lặp, nguồn, và điểm similarity, lưu vào MongoDB.
- FR-4.3: Phản hồi NLP chi tiết (Generative Feedback)
  - ✓ **Mô hình: Google Gemini API**.
  - ✓ **Prompt Engineering:** AI Core Service gửi một Prompt đã cấu trúc hóa đến Gemini, bao gồm:
    - Toàn bộ văn bản bài làm.
    - Rubric đã áp dụng và các điểm số dự đoán của AI.
    - Các đoạn văn bản được đánh dấu là đạo văn (nếu có).
    - **System Instruction:** Hướng dẫn Gemini đóng vai trò là một người chấm điểm học thuật hỗ trợ, cung cấp phản hồi mang tính xây dựng.
  - ✓ **JSON Schema Output:** Yêu cầu Gemini trả về kết quả tuân thủ JSON Schema đã định nghĩa để dễ dàng phân tích và hiển thị. Kết quả được lưu vào trường `ai_report_json` trong DB.

4.6. Thiết kế API và giao tiếp giữa các service

4.6.1. Giao tiếp đồng bộ (Synchronous Communication)

Sử dụng API RESTful qua API Gateway cho các yêu cầu cần phản hồi ngay lập tức hoặc không yêu cầu xử lý nặng:

- User Interface <-> API Gateway:
  - ✓ Authentication/Authorization (Đăng nhập, đăng ký).
  - ✓ CRUD Operations (Quản lý Khóa học, Rubric, Corpus).
  - ✓ Triggering Jobs (Giảng viên nhấn "Chạy AI").

- ✓ Fetching Job Status (Giảng viên kiểm tra trạng thái bài làm).
- Core Service <-> PostgreSQL: Các thao tác đọc/ghi cơ sở dữ liệu giao dịch.

4.6.2. Giao tiếp bất đồng bộ (Asynchronous Communication)

Sử dụng Apache Kafka để xử lý các tác vụ tốn thời gian và tài nguyên, đảm bảo khả năng mở rộng:

- Core Service -> Kafka Topic (`ai_analysis_jobs`): Gửi các yêu cầu phân tích AI mới.
- AI Core Service -> Kafka Topic (`ai_status_updates`): Gửi cập nhật trạng thái của Job (bắt đầu, đang xử lý, hoàn thành, lỗi).
- AI Core Service -> Notification & Sync Service (qua Kafka/DB Event): Thông báo hoàn thành để kích hoạt Grade Passback và gửi thông báo.

Bảng 17.API Endpoint (ví dụ Core Service)

Phương thức	**Endpoint**	**Mô tả**
`POST`	`/api/v1/submissions`	Tải lên bài làm mới (kích hoạt Data Ingestion).
`POST`	`/api/v1/submissions/{id}/analyze`	Kích hoạt phân tích AI cho bài làm (UC-02).
`GET`	`/api/v1/submissions/{id}/status`	Lấy trạng thái xử lý của bài làm.
`PUT`	`/api/v1/submissions/{id}/override`	Hiệu chỉnh điểm/nhận xét thủ công (UC-08, FR-3.2).
`POST`	`/api/v1/submissions/{id}/publish`	Công bố kết quả (UC-04).
`GET`	`/api/v1/results/{token}`	Sinh viên xem kết quả chi tiết (UC-05).

4.7. Thiết kế giao diện người dùng

Mặc dù không trực tiếp là một phần của thiết kế backend, nhưng một UI/UX tốt là yếu tố then chốt để các chức năng của hệ thống có thể được sử dụng hiệu quả.

4.7.1. Giao diện giảng viên (Teacher UI)

- **Dashboard tổng quan:** Hiển thị tổng số khóa học, bài tập đang chờ chấm, trạng thái các job AI gần đây.
- **Quản lý khóa học/bài tập:** Giao diện trực quan để tạo, sửa, xóa, đồng bộ.
- **Cấu hình Rubric (UC-06):** Editor kéo-thả hoặc form dễ sử dụng để định nghĩa tiêu chí và trọng số.
- **Quản lý Corpus (UC-07):** Danh sách tài liệu, nút tải lên, trạng thái xử lý.
- **Xem và Hiệu chỉnh Kết quả (UC-08):**
  - ✓ **Annotated View (UC-05):** Hiển thị văn bản bài làm với các điểm nổi bật:
    - Điểm số từng tiêu chí (của AI và sau hiệu chỉnh).
    - Phản hồi NLP của Gemini được cấu trúc hóa.
    - Các đoạn văn bản được đánh dấu đạo văn (FR-1.3) với nguồn gốc và điểm tương đồng.
    - Form chỉnh sửa điểm và nhận xét với trường `audit_reason` bắt buộc (FR-3.2).
- **Công bố Kết quả (UC-04):** Nút "Công bố" rõ ràng, có xác nhận và hiển thị trạng thái đồng bộ LMS.

4.7.2. Giao diện Sinh viên (Student UI)

- **Trang Kết quả chi tiết (UC-05):**
  - ✓ Thiết kế tối giản, tập trung vào kết quả.
  - ✓ Hiển thị ScoreFinal tổng thể và chi tiết từng tiêu chí.
  - ✓ Phản hồi NLP (FR-4.3) được trình bày dễ đọc, có thể có các tooltip giải thích.
  - ✓ Báo cáo đạo văn: Có thể là một tab riêng hoặc phần cuộn xuống, hiển thị các đoạn trùng lặp và nguồn một cách minh bạch.
  - ✓ Đảm bảo khả năng hiển thị tốt trên thiết bị di động (Responsive Design).

4.7.3. Các nguyên tắc thiết kế UI/UX

- **Dễ sử dụng:** Thiết kế trực quan, giảm thiểu số bước cho các tác vụ phổ biến.
- **Phản hồi rõ ràng:** Thông báo thành công hay thất bại, trạng thái xử lý phải rõ ràng, kịp thời.
- **Nhất quán:** Sử dụng một bộ nguyên tắc thiết kế và thành phần UI nhất quán trên toàn hệ thống.
- **Khả năng truy cập (Accessibility):** Đảm bảo hệ thống có thể được sử dụng bởi người dùng có các nhu cầu đặc biệt.

4.8. Thiết kế cơ sở dữ liệu

Dựa trên chiến lược Polyglot Persistence, chúng ta sẽ thiết kế schema cho PostgreSQL (dữ liệu giao dịch) và định nghĩa cấu trúc cho MongoDB (dữ liệu báo cáo).

4.8.1. PostgreSQL Schema (Transactional & Core Data)

Đây là các bảng chính trong PostgreSQL, đảm bảo tính toàn vẹn và nhất quán của dữ liệu cốt lõi.

Bảng 18. Bảng `users` (Quản lý Tài khoản Giảng viên - UC-09b)

Cột	Kiểu dữ liệu	Ràng buộc	Mô tả
id	UUID	PK	ID duy nhất của người dùng.
email	VARCHAR(255)	UNIQUE, NOT NULL	Email đăng nhập.
password_hash	VARCHAR(255)	NOT NULL	Mật khẩu đã băm.
first_name	VARCHAR(255)	NOT NULL	Tên.
last_name	VARCHAR(255)	NOT NULL	Họ.
lms_user_id	VARCHAR(255)	NULL	ID người dùng từ LMS (nếu có tích hợp).
created_at	TIMESTAMP	NOT NULL	Thời gian tạo tài khoản.
updated_at	TIMESTAMP	NOT NULL	Thời gian cập nhật cuối.

Bảng 19. Bảng `lms_integrations` (Cấu hình Tích hợp LMS - UC-10)

Cột	Kiểu dữ liệu	Ràng buộc	Mô tả
id	UUID	PK	ID duy nhất của cấu hình tích hợp.
user_id	UUID	FK users.id	Giảng viên sở hữu cấu hình này.

lms_type	VARCHAR(50)	NOT NULL	Loại LMS (e.g., 'CANVAS', 'MOODLE').
lms_base_url	VARCHAR(255)	NOT NULL	URL gốc của LMS.
client_id	VARCHAR(255)	NOT NULL	Client ID cho OAuth/API Key.
client_secret_ref	VARCHAR(255)	NOT NULL	Tham chiếu đến Secret Manager (không lưu trực tiếp secret).
access_token_ref	VARCHAR(255)	NULL	Tham chiếu Access Token (nếu OAuth - không lưu trực tiếp).
is_active	BOOLEAN	DEFAULT TRUE	Cấu hình có đang hoạt động không.
created_at	TIMESTAMP	NOT NULL	Thời gian tạo cấu hình.
updated_at	TIMESTAMP	NOT NULL	Thời gian cập nhật cuối.

Bảng 20. Bảng `courses` (Quản lý Khóa học - UC-09a)

Cột	Kiểu dữ liệu	Ràng buộc	Mô tả
id	UUID	PK	ID duy nhất của khóa học trong HouScore-AI.
lms_id	VARCHAR(255)	UNIQUE NULL	ID Khóa học từ LMS (nếu có đồng bộ).
user_id	UUID	FK users.id	Giảng viên tạo/quản lý khóa học.
name	VARCHAR(255)	NOT NULL	Tên khóa học.
description	TEXT	NULL	Mô tả khóa học.
created_at	TIMESTAMP	NOT NULL	Thời gian tạo.
updated_at	TIMESTAMP	NOT NULL	Thời gian cập nhật cuối.

Bảng 21. Bảng `rubrics` (Quản lý Rubric - UC-06)

Cột	Kiểu dữ liệu	Ràng buộc	Mô tả
id	UUID	PK	ID Rubric.
user_id	UUID	FK users.id	Giảng viên tạo Rubric.
name	VARCHAR(255)	NOT NULL	Tên Rubric.
is_public	BOOLEAN	DEFAULT FALSE	Rubric có công khai cho người khác không.
created_at	TIMESTAMP	NOT NULL	Thời gian tạo.
updated_at	TIMESTAMP	NOT NULL	Thời gian cập nhật cuối.

Bảng 22. Bảng `rubric_criteria` (Chi tiết Rubric - UC-06)

Cột	Kiểu dữ liệu	Ràng buộc	Mô tả
id	UUID	PK	ID tiêu chí.
rubric_id	UUID	FK rubrics.id	Rubric cha.
criterion_name	VARCHAR(255)	NOT NULL	Tên tiêu chí.
weight	DECIMAL(5,2)	NOT NULL	Trọng số của tiêu chí (0.00 - 1.00, tổng phải = 1.00).
order_index	INT	NOT NULL	Thứ tự hiển thị.

8.1.6. Bảng `submissions` (Tải bài làm & Quy trình AI - UC-01, UC-02, UC-08, UC-04, UC-05)

Cột	Kiểu dữ liệu	Ràng buộc	Mô tả
id	UUID	PK	ID bài làm.

course_id	UUID	FK courses.id	Khóa học liên quan.
student_lms_id	VARCHAR(255)	NOT NULL	ID Sinh viên từ LMS (đảm bảo duy nhất).
rubric_id	UUID	FK rubrics.id	Rubric được áp dụng.
file_path_s3	VARCHAR(512)	NOT NULL	Đường dẫn đến tệp thô trong S3/GCS.
status	ENUM	NOT NULL	Trạng thái: 'UPLOADED', 'PROCESSING', 'ANALYZED', 'OVERRIDDEN', 'PUBLISHED', 'ERROR'.
ai_score_raw	DECIMAL(5,2)	NULL	Điểm thô từ AI (FR-1.2) - Giữ để đối chiếu.
final_score	DECIMAL(5,2)	NULL	Điểm cuối cùng (sau khi hiệu chỉnh) - FR-4.1.
ai_report_mongo_id	VARCHAR(255)	NULL	ID của báo cáo chi tiết AI trong MongoDB (FR-4.3).
plagiarism_report_mongo_id	VARCHAR(255)	NULL	ID của báo cáo đạo văn trong MongoDB (FR-1.3).
published_at	TIMESTAMP	NULL	Thời gian công bố (FR-4.1).
created_at	TIMESTAMP	NOT NULL	Thời gian nộp bài.
updated_at	TIMESTAMP	NOT NULL	Thời gian cập nhật cuối.

Bảng 23. Bảng `audit_logs` (Kiểm tra Truy vết - FR-3.2, UC-08)

Cột	Kiểu dữ liệu	Ràng buộc	Mô tả
id	UUID	PK	ID bản ghi log.

user_id	UUID	FK users.id	Giảng viên thực hiện hành động.
submission_id	UUID	FK submissions.id	Bài làm bị tác động.
action_type	VARCHAR(50)	NOT NULL	Loại hành động: 'SCORE_OVERRIDE', 'FEEDBACK_EDIT', 'STATUS_CHANGE'.
old_value	TEXT	NULL	Giá trị cũ trước khi thay đổi.
new_value	TEXT	NULL	Giá trị mới sau khi thay đổi.
reason	TEXT	NOT NULL	Lý do bắt buộc cho hành động (UC-08, FR-3.2).
timestamp	TIMESTAMP	NOT NULL	Thời gian (UTC) xảy ra hành động.

4.8.2. MongoDB Schema (Flexible & Report Data)

MongoDB sẽ lưu trữ các tài liệu JSON phức tạp, không cần schema cố định, cho phép AI Core Service linh hoạt trong việc tạo báo cáo.

- ***Collection `ai_reports` (FR-4.3)***
  - ✓ Mục đích: Lưu trữ đầu ra chi tiết của Google Gemini và các phân tích AI khác cho từng bài làm.
  - ✓ Cấu trúc ví dụ (đã định nghĩa trước):

    JSON

```
{
    "_id": ObjectId("..."), // mongo_id, được tham chiếu từ
submissions.ai_report_mongo_id
    "submission_id": "UUID_SUB_XYZ",
    "overall_summary": "Bài làm cho thấy sự hiểu biết lý thuyết
tốt...",
    "rubric_feedback": [
        { "criterion_name": "...", "ai_comment": "...",
"score_predicted": ... },
        // ...
    ],
    "nlp_analysis": {
        "coherence_score": 85,
        "tone_suggestion": "...",
        "structural_recommendation": "..."
    },
```

```
    "generated_at": ISODate("...")
}
```

- ***Collection `plagiarism_reports` (FR-1.3)***
  - ✓ Mục đích: Lưu trữ kết quả phát hiện đạo văn ngữ nghĩa chi tiết.
  - ✓ Cấu trúc ví dụ:

    JSON

```
{
    "_id": ObjectId("..."), // mongo_id, được tham chiếu từ
submissions.plagiarism_report_mongo_id
    "submission_id": "UUID_SUB_XYZ",
    "semantic_overlap_percentage": 5.2,
    "risk_level": "LOW",
    "matched_segments": [
        {
            "text_segment": "Đoạn văn bản bị trùng lặp.",
            "source_type": "CORPUS", // hoặc "CROSS_SUBMISSION"
            "source_ref": "Document_ID_ABC" // hoặc
"Submission_ID_123"
            "cosine_similarity": 0.92,
            "start_char": 100,
            "end_char": 250
        },
        // ...
    ],
    "analysis_date": ISODate("...")
}
```

4.9. Thiết kế bảo mật chi tiết

Xây dựng trên nền tảng đã có, chúng ta sẽ đào sâu hơn vào các cơ chế bảo mật.

4.9.1. Xác thực và Ủy quyền (Authentication & Authorization)

- Xác thực:
  - ✓ Giảng viên: Ưu tiên OpenID Connect (OIDC) hoặc OAuth 2.0 với các IdP (Identity Providers) như Google, Microsoft, hoặc tích hợp trực tiếp với LMS thông qua LTI/OAuth. Nếu không, sử dụng xác thực Email/Mật khẩu truyền thống với BCrypt hashing cho mật khẩu.
  - ✓ Sinh viên: Truy cập thông qua Secure One-Time Link (JWT) hoặc LTI Launch từ LMS.
- Ủy quyền (Authorization):
  - ✓ RBAC (Role-Based Access Control): Giảng viên có quyền `admin` hoặc `teacher`. Sinh viên có quyền `student`.

- ✓ Scoped Access: Giảng viên chỉ có thể quản lý các khóa học, rubric mà họ sở hữu hoặc được gán quyền. Sinh viên chỉ có thể xem kết quả bài làm của chính họ.
- ✓ Policy Enforcement: API Gateway sẽ thực thi các chính sách ủy quyền cơ bản. Core Service sẽ thực hiện kiểm tra quyền chi tiết hơn ở cấp độ nghiệp vụ.

4.9.2. Bảo vệ Dữ liệu (Data Protection)

- Mã hóa dữ liệu khi lưu trữ (Encryption At Rest - NFR-SEC-2):
  - ✓ Object Storage (S3/GCS): Sử dụng Server-Side Encryption (SSE-S3/GCS-managed key) hoặc Customer-Managed Keys (CMK) để mã hóa tệp bài làm thô và Corpus.
  - ✓ PostgreSQL/MongoDB: Kích hoạt mã hóa ổ đĩa (Volume Encryption) trên các máy chủ cơ sở dữ liệu.
- Mã hóa dữ liệu khi truyền tải (Encryption In Transit):
  - ✓ Giao tiếp Client-Server: Bắt buộc sử dụng HTTPS/TLS 1.2+ với chứng chỉ được cấp bởi CA đáng tin cậy.
  - ✓ Giao tiếp Microservice-to-Microservice: Triển khai mTLS (Mutual TLS) sử dụng Service Mesh (ví dụ: Istio trên K8s) hoặc cấu hình thủ công để xác thực lẫn nhau và mã hóa tất cả lưu lượng truy cập nội bộ.
- Quản lý bí mật (Secret Management):
  - ✓ Sử dụng HashiCorp Vault hoặc AWS Secrets Manager/Google Secret Manager để lưu trữ tất cả các thông tin nhạy cảm (API Keys, Client Secrets, Database Credentials).
  - ✓ **`client_secret_ref`** và **`access_token_ref`** trong **`lms_integrations`**: Các cột này sẽ chỉ chứa một ID/khóa để truy xuất bí mật thực sự từ Secret Manager trong thời gian chạy (runtime), KHÔNG phải là giá trị bí mật trực tiếp.

4.9.3. Ghi Log và Giám sát Bảo mật (Security Logging & Monitoring)

- Audit Trail (FR-3.2, NFR-SEC-4):
  - ✓ Tất cả các hành động nhạy cảm (Đăng nhập, Thay đổi điểm, Thay đổi cấu hình Rubric, Xóa dữ liệu) phải được ghi vào bảng `audit_logs` (PostgreSQL) với đầy đủ thông tin (ai, khi nào, cái gì, tại sao - `reason`).
  - ✓ Bản ghi audit không thể sửa đổi (Immutable).

- Log Aggregation: Thu thập log từ tất cả các Microservices vào một hệ thống tập trung (ví dụ: ELK Stack, Grafana Loki) để dễ dàng giám sát, phân tích và phát hiện bất thường.
- Security Monitoring: Thiết lập cảnh báo cho các sự kiện bảo mật quan trọng (ví dụ: đăng nhập thất bại liên tục, truy cập trái phép, lỗi hệ thống).

4.10. Lựa chọn công nghệ

Dựa trên các yêu cầu và thiết kế đã nêu, đây là các đề xuất công nghệ cụ thể:

Thành phần/Microservice	Công nghệ Lựa chọn	Lý do
API Gateway	Kong API Gateway	Mã nguồn mở, hiệu suất cao, hỗ trợ plugin mạnh mẽ (Auth, Rate Limiting).
Core Service	Laravel (PHP) + Lumen	Nhanh chóng phát triển API, cộng đồng lớn, dễ bảo trì, Lumen tối ưu cho Microservices.
AI Core Service	Python + FastAPI / Flask	Python là chuẩn cho AI/ML. FastAPI/Flask nhẹ, nhanh, dễ tích hợp với PyTorch/TensorFlow.
Data Ingestion	Go (Golang)	Hiệu suất xử lý I/O cao, đồng thời tốt, phù hợp cho việc đọc/ghi tệp nhanh.
Notification & Sync	Go / Node.js (NestJS)	Go cho hiệu suất cao, xử lý sự kiện; Node.js/NestJS cho phát triển nhanh, xử lý bất đồng bộ tốt.
Messaging Queue	Apache Kafka	Nền tảng phân tán, bền vững, hiệu suất cao, phù hợp cho luồng dữ liệu lớn và xử lý bất đồng bộ.
Relational DB	PostgreSQL	Mạnh mẽ, đáng tin cậy, hỗ trợ ACID, có PgVector cho Vector DB nếu cần.

NoSQL DB (Reports)	MongoDB	Linh hoạt cho schema bán cấu trúc, dễ dàng lưu trữ các báo cáo JSON phức tạp của AI.
Vector DB	Pinecone / PgVector (FAISS)	Pinecone: Giải pháp chuyên dụng, managed, hiệu suất cao cho ANN. PgVector: Tích hợp với Postgres, đơn giản hơn cho khởi đầu hoặc quy mô nhỏ hơn.
Object Storage	AWS S3 / Google Cloud Storage	Managed, bền vững, khả năng mở rộng không giới hạn.
Secret Manager	HashiCorp Vault / AWS Secrets Manager	Bảo mật mạnh mẽ, quản lý vòng đời của bí mật.
Deployment Platform	Kubernetes (GKE/EKS)	Quản lý container, tự động hóa triển khai, tự động mở rộng, hỗ trợ GPU Node Pools.
Monitoring/Logging	Prometheus/Grafana + ELK Stack (Elasticsearch, Logstash, Kibana)	Giải pháp mạnh mẽ để thu thập metrics, log và hiển thị dashboard.
Frontend Framework	React / Vue.js	Phát triển UI tương tác, hiệu suất cao.
AI/ML Models	Google Gemma (Embeddings)	Tạo Vector Embedding chất lượng cao.
AI/ML Models	Google Gemini (Generative AI)	Tạo phản hồi NLP chi tiết, tận dụng khả năng suy luận mạnh mẽ.